What Post-Training Repair Buys on llama.cpp Quantization: A Paired Audit on and off the Calibration Distribution

Max Khan
OpenBeast project · github.com/MaximilianKhan/openbeast
October 2026

Abstract

We measure what post-training repair of low-bit llama.cpp quantization buys, on five Qwen-family models and Llama-3.1-8B, on one GPU. The main comparisons are scored against BF16 references on the calibration distribution and on two corpora that no calibration touched. (1) Re-rounding re-chooses a Q2_K file's codes under a captured activation-covariance metric (GPTQ's sweep on llama-quantize's frozen grid). It lowers KL divergence (KLD) by a third at 0.8B. At 8B and 27B, the registered recipe (calibration mixing prose and code) is worse than the untouched file on wikitext and FineWeb-Edu, and a wikitext-only calibration gains at most 4% on wikitext while damaging both held-out corpora. At 0.8B, at 8B and at 27B, re-rounding damages kinds of text its calibration does not cover, as concurrent work reports for generic quantization. Letting GPTQ refit the grid beats freezing it in eleven of twelve cells. (2) Symmetric whitened low-rank correction improves its base on every corpus we scored; off the calibration distribution the gain is smaller in the ladder and recovery comparisons (0.25 to 0.96 of the wikitext gain, depending on model and corpus) and 0.84 to 1.05 of it on the head-to-head files (§5.3). Against the stock types it falls short: an I-quant type at fewer bytes beats it on every corpus at 8B and 27B. At 27B its 4.5% lead over the adjacent K-quant, resolved on the whole wikitext test set, appears only on the calibration distribution, and a same-byte file mixing stock types beats it on every corpus at 27B and by a factor of three at 0.8B. (3) Against released GSQ-RCO checkpoints of Qwen3.8-27B, KLD to the reference and likelihood of the text disagree within one 2,048-token run on wikitext: the trained files are further from BF16 at both size classes and, where the likelihood difference is resolved (the IQ3 class), assign the text higher likelihood, at the IQ3 class as high as the BF16 model itself; for these files KLD measures fidelity, and lower perplexity does not measure quality either. [PENDING:F — capability replication.] (4) Fused adapter kernels raise 0.6B decode by 79% and 27B decode by 2.7% [E36, E14, E27].

1. Introduction

1.1 The setting

A consumer GPU such as the RTX 5090 pairs a fast processor with 32 GB of memory, and the models people want to run on it are several times larger than that at training precision. Single-stream decoding is limited by memory bandwidth, because every token reads the full weight set. We measured the consequence on one 27B model, Qwen3.6-27B (the checkpoint is heretic-v2, an uncensored community finetune of Qwen3.6-27B that keeps the base model's multi-token-prediction layers): the Q6_K file occupies 23.6 GB of VRAM and decodes at 61 tok/s, and the Q2_K file occupies 13.0 GB and decodes at 99.7 tok/s [rollup]. On this card, for these two files, the smaller one was also the faster one. We take that as the motivation for the study and not as a general result.

The question is how much quality post-training processing can recover per byte, or at zero added bytes, on top of a mature quantization stack. We ask it on llama.cpp, whose family of imatrix-weighted block quantizers (the K-quant and I-quant types, nominally about 1.5 to 6.5 bits per weight; terms are defined in §2) is the strongest deployed baseline in this regime and a harder rival than round-to-nearest. The models are five Qwen-family models and Llama-3.1-8B, all measurements come from one GPU, and the main comparisons are scored on the calibration distribution and on two corpora that no calibration touched.

1.2 What this paper does not claim

It does not claim that low-rank factorization compresses these models. A singular-value census of every tensor in our smallest model finds that capturing 95% of unweighted Frobenius energy needs 50-80% of full rank on every projection type [E1]. The quantization residual R = W − dequant(Q), the object that the error-correction lineage

corrects (ZeroQuant-V2/LoRC, [1]; LQER, [2]; QERA, [3]; EoRA, [4]; CALDERA, [5]), is less structured still: 90% of its energy needs about 62% of full rank [E3]. Both observations agree with the published account (LQER; RILQ, [6]).

It does not claim a method that beats llama.cpp's stock quantization types. At 27B the corrected configuration is ahead of Q3_K_S by 0.0047 KLD (5%, t = −2.62 on 100 chunks; 4.5%, t = −4.32 on the whole 580-chunk test set) on the calibration distribution, behind it on FineWeb-Edu (t = +3.26) and unresolved on code; a same-byte file that mixes stock types and has no adapter is ahead of it on all three corpora, and the I-quant type IQ3_XS is better than both at fewer bytes (§4.3). At 0.8B the same-byte mixed-type control has a third of the corrected configuration's KLD [E36].

1.3 Contributions

(1) Re-rounding on GGUF grids: a small-model effect, and what it needs. llama-quantize chooses a grid per block and then rounds each element independently. We keep the grid bytes as shipped and re-choose only the codes under a captured activation-covariance metric, using the GPTQ sweep ([7]); the output is a standard GGUF of identical size. On Qwen3.5-0.8B at Q2_K it lowers KL divergence by 33% (t = −28.4, 40/40 chunks). On Llama-3.1-8B and Qwen3.6-27B the registered recipe, calibrated on a mix of prose and code, is worse than the untouched file on wikitext and FineWeb-Edu, and a wikitext-only calibration gains under 4% on wikitext while damaging both held-out corpora. At 27B this happens although the tensors the sweep covers carry 76-80% of the file's divergence; on three tensors we checked independently, the sweep halves the error under the calibration metric while raising the unweighted error by 16-25% (§3.1). A grid of five calibration texts against three evaluation corpora at 0.8B shows where the gain lands when there is one: on the kinds of text the calibration covers, with damage to the kinds it does not. Concurrent work reports the same reversal for frozen-grid code refinement in generic quantization ([8]); we measure it on llama.cpp's deployed K-quant grid, against the file as written, with code as the uncovered kind. Letting GPTQ refit the grid inside the same format is better than freezing it in eleven of twelve cells (§3.4), so the frozen grid is a restriction and not a feature. The earlier 27B result of this project (−13.3%) came from a legacy-provenance pair and does not reproduce under single-step provenance. Related methods are GSQ ([9]), ReQuant ([10]) and SchurQuant ([11]); the relation is stated in §8.

(2) A paired audit of low-rank correction on llama.cpp's own formats, on and off the calibration distribution. We measure symmetric whitened low-rank correction (QERA's closed form) at four dense sizes in two families and on one mixture-of-experts model, against the stock types and against mixed-type controls built at the same bytes at 0.8B, 8B, 27B and on the mixture-of-experts model. The correction improves its own base everywhere we measured it, with a recovered fraction that grows as the base gets thinner (7%, 12% and 32% at Q2_K, IQ2_XS and IQ1_S at 27B, with the recipe fixed), and that is smaller off the calibration distribution: the held-out gain is 0.42-0.47 of the wikitext gain at Qwen3.6-27B, 0.25-0.69 at Llama-3.1-8B and 0.59-0.96 at 0.8B (§4.3, §4.5). On Qwen it loses to the control everywhere one was built; on Llama-3.1-8B it beats the adjacent K-quant type on both prose corpora. It never beats the I-quant type below it in bytes at 8B or 27B (§4.3). Three placements of the low-rank bytes cannot be told apart at 27B, within a bound we state (§4.4). Fixed-rank capture is sublinear in rank and, across a single width step, falls more slowly than r/d predicts (§4.2). Several of the audit's negative results are its best-resolved findings (§4.7, Appendix A). The same-byte question was posed informally upstream (ik_llama.cpp #15); this is its controlled version. A stronger, asymmetric compensator ([12]) was not run against the controls.

(3) A head-to-head with released trained checkpoints, and what KL divergence measures. Against the released GSQ-RCO checkpoints of Qwen3.8-27B, the untrained vendor baseline at matched bytes is closer to the BF16 distribution on wikitext at 512 and at 2,048 tokens, while the trained checkpoints assign wikitext higher likelihood in the same 2,048-token run (resolved at the IQ3 class). At the IQ3 class the trained file's likelihood of wikitext is level with the BF16 model's own. For these files KL divergence to the reference ranks fidelity to it and not likelihood of the text, and neither is a measure of capability. That KL divergence measures displacement and not direction has been argued before ([13]); this is a paired, matched-byte instance on released files. We argue it is the right instrument for a method that only approximates the reference and an unreliable one for ranking a refined artifact against an untrained one (§5.4). Our correction (+0.9 GB, not byte-matched) moves the baseline closer to the reference on every corpus and context we measured, without a resolved NLL gain at the IQ3 class, and a stock file at near-equal total bytes beats it. [PENDING:F — capability replication, three runs per arm.]

(4) Serving kernels. llama.cpp's stock adapter path costs 20-40% of decode speed. A fused-kernel series removes most of that at 0.6B (+79%, one same-session pair) and gives +2.7% at 27B (N = 10, interleaved). A separate allocator fix raised base decode at 0.6B by 19.6% in the better of two sessions and 11% in the other, with no measurable change at 27B (§6) [E14, E25].

The measurement protocol these results follow (single-step provenance, paired per-chunk statistics, held-out corpora, same-byte controls, pre-registration) is part of the method and is stated in §2.5.

1.4 Relation to prior work

The pipeline is assembled from published mathematics: QERA's whitened closed forms, LoftQ's alternation, and GPTQ's sweep on a frozen grid. What we add is where it is measured: on llama.cpp's deployed formats, against its stock types and a same-byte mixed-type control, with paired statistics. §8 gives the comparison in full.

The paper follows the pipeline: method (§2), re-rounding (§3), the correction audit (§4), the GSQ-RCO head-to-head (§5), serving (§6), limitations (§7), related work (§8). Appendix A holds additional negative results.

2. Method

The pipeline has four stages: capture second-moment statistics on calibration text (§2.1); fit whitened corrections in closed form (§2.2) or re-round the base's codes on their frozen grids (§2.3); optionally alternate quantizer and corrector (§2.4); serve through llama.cpp's adapter path or our fused kernels (§6). Quality claims follow the measurement protocol of §2.5.

Terms. The *imatrix* is llama.cpp's importance statistic: for each weight column, the mean squared input activation on calibration text, which llama-quantize uses to weight its rounding error. *K-quants* (Q2_K, Q3_K_S, Q3_K_M, Q4_K_M, Q6_K) are llama.cpp's block-wise scalar quantizers; the digit is the nominal bit width of the main tensor type and the suffix (S, M) says how many tensors are promoted to a wider type. *I-quants* (IQ1_S, IQ2_XS, IQ3_XXS, IQ3_XS) are its codebook-based types at similar or smaller sizes. We call the set of stock types ordered by file size *the ladder* and one stock type *a rung*. *UD* marks Unsloth Dynamic files, vendor quantizations that choose a type per tensor. *MTP* is the multi-token-prediction head some of these checkpoints carry; we pin its tensors to a fixed type. *GDN* is Gated DeltaNet, the linear-attention layer of the hybrid architecture (llama.cpp name `qwen35`) used by Qwen3.5-0.8B and Qwen3.8-27B. The sizes of the types used at 27B, each one quantization step from BF16 [E27]:

type	file GB (Qwen3.6-27B)	bits per weight (file bytes × 8 / parameters)
Q2_K	11.00	3.22
IQ3_XXS	11.48	3.36
IQ3_XS	12.26	3.59
Q3_K_S	12.37	3.62
Q3_K_M	13.59	3.98
Q4_K_M	16.84	4.93

A nominal "Q2_K" file is above 3 bits per weight because llama-quantize promotes some tensors to wider types. Bits per weight in this paper are file bytes over parameter count unless marked nominal.

Configurations. We use four names throughout. *Re-rounding* is the code re-selection of §2.3. *Full-covariance correction* is the low-rank adapter of §2.2 whitened by the full input Gram, as opposed to the imatrix diagonal. *The mixed base* is the 27B base with FFN tensors at Q3_K and the remaining tensors at Q2_K. *The corrected configuration* is a base plus its correction; at 27B it means the mixed base plus a rank-128 full-covariance correction with Q8_0 factors on the non-FFN tensors. *The control* is a file with no adapter, built from stock tensor types at the corrected configuration's byte count.

2.1 Statistics capture

Input-side Grams. llama-imatrix accumulates $\mathrm{diag}(E[x^2])$ per weight column, which is the diagonal scaling used by LQER ([2]) and QERA-approx ([3]). We extended it (a local, environment-gated C++ patch of about 140 lines) to accumulate the full Gram $G = \Sigma\, x\, x^T$ per distinct layer input: one for the attention q/k/v input, one for the FFN gate/up input, one each for attn_output and ffn_down. Accumulation is fp32. The diagonal of the captured

Gram matches the stock imatrix to a median relative error of 2e-8, and the matrices are symmetric and positive semidefinite [E10]. Inputs wider than 8,192 dimensions are captured block-diagonally in 8 blocks, an approximation forced by memory whose consequence is stated in §2.2.

Mixture-of-experts extension. The expert-routed branch needs its own accumulator, because rows from different experts are mixed and must be unpacked per (token, slot) pair. We capture one pooled Gram per expert stack, the union of routed inputs, and not a Gram per expert: at 16 calibration chunks there are about 256 samples per expert for a 2,048-dimensional Gram, which is rank-deficient, while the pooled Gram is full-rank. The validation gates pass on Qwen3.6-35B-A3B [E23].

Output-gradient Grams. For two-sided metrics we built llama-gradmatrix, which runs llama.cpp's training loop forward and backward with learning disabled and captures $T = \Sigma\, g\, g^T$ per tensor, where g is the loss gradient at the output row. Two repairs were needed. llama.cpp's trainer does not backpropagate through the KV cache (upstream issue #21037 reports the same gap), which we bypass by feeding k/v directly to attention when the window is one micro-batch. Several operations of the GDN layers have no backward pass, which we handle with an explicit gradient cut that stops flow through those layers' token mixing and keeps the residual path. At 0.8B the cut biases the 18 linear-attention layers, which matters in §3.2 [E20].

2.2 Whitened low-rank correction

Given the residual R = W_ref − dequant(Q) and a whitener L (the damped Cholesky factor of G/count; damping 0.01 × mean diagonal, the convention of [7] and [14]; the route is numerically viable at this size by [15]), the rank-r corrector minimizes the activation-weighted error $\|(R - \Delta)\, L\|_F^2$. The two-sided weighted problem is NP-hard ([16]); the one-sided problem has a closed form: whiten, truncate, unwhiten. It is the $L_g = I$ case of the two-sided objective of OBD-LLM ([17]). We stay one-sided for three reasons: a tool that starts from a shipped file has no gradients; a diagonal output weighting provably cannot change a frozen-grid rounding decision (§2.3); and the published two-sided gain is on weight decomposition at 8B, not on residual correction of K-quant grids [theory-L6].

We use the projection form: B = U_r, the top-r left singular vectors of R L, and $A = B^T R$. For any invertible right whitener the unwhitening factor cancels, so this is the exact optimum; our code check agrees with the explicit argmin to 7e-15 [review]. Factors are computed by randomized SVD ([18]; oversampling 32, 4 power iterations).

The blocked-whitening caveat. For inputs above 8,192 dimensions the captured Gram is block-diagonal. At 27B this applies to ffn_down, whose input has 17,408 dimensions (8 blocks of 2,176). The projection form is then optimal for the block-diagonal surrogate and not for the true $E[xx^T]$; a synthetic test with cross-block correlation measured a 25% excess of true-metric loss for the blocked optimum [review]. Every "full-covariance" 27B row therefore means full covariance for inputs up to 8,192 dimensions and 8-block-diagonal for ffn_down. End-to-end measurements score the artifact and are unaffected; the mechanism claim that the metric was the bottleneck is untested against a truly full metric at that width.

Two-sided variant. With both S and T captured, $B = L_T^{-T}\, U_r$ and A is solved against L_S, with a factor-balancing step that leaves the product unchanged and prevents half-precision overflow [review].

Factor density. Factors are stored as Q8_0 (llama.cpp's 8-bit block format), which halves their bytes at measured quality parity (rank 64 with Q8_0 factors: perplexity 31.26 against 31.29 with F16 factors at 0.6B), the observation of [19] and [1] confirmed in this pipeline [journal].

Serving and what is not stock. The corrected model is a quantized base GGUF plus a LoRA-form adapter GGUF, and $y = Q{\cdot}x + s{\cdot}B{\cdot}(A{\cdot}x)$ is llama.cpp's stock adapter path. The artifact therefore serves on an unmodified llama-server, at the decode cost §6 measures. Capture is not stock: every full-covariance number depends on our Gram patch to llama-imatrix, and gradient Grams on llama-gradmatrix.

2.3 Re-rounding on frozen grids

A K-quant tensor stores, per 16-element sub-block, a scale and offset (the grid) and a low-bit code per element. llama-quantize chooses both and rounds each element independently. We keep the grid bytes exactly as written and re-choose only the codes to minimize the whitened objective of §2.2 with $\Delta = 0$, which is a closest-vector problem on the frozen grid. The solver is the GPTQ error-feedback sweep: process columns in order, round the current column

on its grid, and propagate the scaled error to the unprocessed columns through the upper-triangular Cholesky factor of the inverse Gram. With frozen grids the objective does not increase from sweep to sweep ([20]).

The objective needs W_ref, the higher-precision weights, and the Gram, which is captured on the patched binary. The method therefore starts from a model's source weights plus a calibration pass, and writes a file that is byte-compatible with the stock one: same size, same type tags, loadable by any llama.cpp build.

Two related methods work the same quadratic. ReQuant ([10]) performs cyclic coordinate descent on it outside the GGUF format, and SchurQuant ([11]) also refits the scales [theory-L6].

Implementation points that affect the numbers: the Q2_K codec was mirrored from gguf-py and round-trips real tensors byte-identically; the triangular convention matters (a lower factor disables the feedback and reproduces llama-quantize's codes exactly); and Cholesky failures escalate damping [E13]. Only the Q2_K codec is implemented. A two-sided (Kronecker) generalization of the sweep is derived and tested in the experiment record, together with a small negative result: a diagonal output-side weighting cannot change any code, because each row's argmin is scale-invariant [E24].

2.4 Alternation

Correction quality depends on the base, and the base's grid choices depend on what the corrector will absorb. We alternate Q_{t+1} = quant(W − C_t) and C_{t+1} = corr(W − Q_{t+1}), which is LoftQ's template ([21]) with llama-quantize as a black-box quantizer and the whitened closed form as the corrector, and we keep the best iterate and stop when a round is worse. Because the quantizer's output set is finite and the corrector step is an exact minimum, the best-iterate value is non-increasing and becomes constant after finitely many rounds. In practice two or three rounds are used and later changes are below noise. The argument, its measured amendment (iterates settle onto a small limit cycle and not a fixed point), and the pointers to related metric-modification schemes are in Appendix A.5 [theory-alt].

2.5 Measurement protocol

Results in §3-§6 follow these rules or are labeled legacy [protocol].

- **Reference and provenance.** The reference is BF16 where it exists. For the mixture-of-experts model of §4.6 we had no BF16 and used the vendor UD-Q4_K_M file, stated at the table. Every base and adapter derives from the reference in one quantization step; rows of other provenance are labeled legacy and are not tabled beside single-step rows without a provenance label.
- **Build.** Every row of §3.1, §4.3, §4.5 and §5.4 is scored on llama.cpp b10865 against reference logits generated on that build. An earlier draft of this study scored the hybrid-architecture models on builds that predate llama.cpp's Gated-DeltaNet normalization fix (b10829), which changes their forward pass; rows from that build are paired only with each other and are named where quoted.
- **Corpora.** Calibration is wikitext-2 train unless stated. Evaluation uses three corpora: wikitext-2 test; FineWeb-Edu (the first 400 documents of the sample-10BT split); and a 400 KB code corpus disjoint from the code used in any calibration. A FineWeb-Edu calibration slice, where used, comes from documents outside the evaluation slice, and no calibration text shares a 200-byte window with an evaluation text.
- **Metrics.** Mean KL divergence (KLD) and top-1 agreement against reference logits at a 512-token context (and at 2,048 tokens in §5.4), on at least 40 chunks (100 for the 27B comparisons of §4.3 and §4.5, and all 580 chunks of the wikitext-2 test set for the whole-test-set rows of §4.3; 20 for the mixture-of-experts table and some legacy rows), with standard errors. KLD is used because it tracks answer flips ([22]) and is the measure llama.cpp's maintainers use (discussion #4110). Full-corpus perplexity is secondary. Both measure closeness to the reference or to the text, not capability (§5.4, §7.1).
- **Paired statistics.** Compared files share evaluation tokens and reference logits. A comparison is decided by the per-chunk difference: its mean, standard error and t. We write t for a paired statistic with n − 1 degrees of freedom, where n is the chunk count given with it, and we reserve "standard errors" for unpaired distances, labeled as such.
- **What the t statistics are conditional on.** All intervals are conditional on one calibration sample and one evaluation corpus; they quantify evaluation-chunk variance only. Calibration-sampling variance, measured with three disjoint calibration samples at 0.8B, moves the re-rounding gain by at most 0.018 KLD (§3.1).

- **Multiplicity.** No family-wise correction was applied across the paper's contrasts, so margins near |t| = 2-3 are suggestive and not confirmatory.
- **Dependence between chunks.** Adjacent 512-token chunks of one corpus are not independent. For the load-bearing pairs we report, in addition to t, an exact sign test, a Wilcoxon signed-rank test, and a bootstrap over contiguous blocks of 10 chunks (at n = 40 that is four blocks, too few to lean on) [E35].
- **Byte-fair comparison.** Total bytes are base plus adapter. A comparison against the ladder includes K-quants, I-quants, and where built a mixed-type control at the matched byte count.
- **Speed.** Same-session A/B pairs only. Between-session drift on identical configurations reached 9%.
- **Pre-registration.** From the second round of internal review onward, each experiment's configuration, pairs and decision rules were written to a timestamped experiment log before the run. The log is our own record (it ships with the supplementary material) and has not been audited externally. The experiments added in the final round were registered in one plan, committed to version control before any of them was scored; where a registered rule could not be applied as written, the amendment was recorded before the run and is stated beside the result. The times written inside the plan are when each section was drafted; the commit times, which are 15-70 minutes later, are the ones that precede the runs.
- **Claims.** We call a relation a law only with at least three scale points and a stated functional form; otherwise it is an observation.

3. Results I: re-rounding at zero added bytes

3.1 What re-rounding changes, and what it does not

Re-rounding changes no file size, VRAM use or serving speed: it re-chooses which available code each weight gets, using one calibration pass and the model's higher-precision source weights. We report it at four sizes and on two families. It improves the file at 0.6B and 0.8B; at 8B and 27B it does not, and at every size where we scored held-out text (0.8B, 8B, 27B) it damages text its calibration does not cover.

Qwen3-0.6B (Q2_K with imatrix; legacy scoring against a Q8_0 reference). [E13]

file (296 MB)	PPL	KLD	top-1
Q2_K, as written by llama-quantize	43.33	0.766	58.2%
Q2_K, re-rounded	35.54	0.556	64.0%

Perplexity falls by 18%. The two rows are 15.6 unpaired standard errors apart on perplexity and about 19 on KLD [E35]. No per-chunk logs exist for this pair, and no smaller stock rung was scored against it.

Qwen3.5-0.8B (Q2_K; BF16 reference; three corpora; current build). A second, hybrid-attention architecture. Every file is one quantization step from BF16 with a 40-chunk wikitext-2-train imatrix and Gram capture; KLD is on 40 chunks of each corpus against BF16 logits generated on llama.cpp b10865 [E36]:

file	MB	KLD, wikitext	KLD, FineWeb-Edu	KLD, code
IQ2_M, stock	392	0.5477	0.5391	0.4881
IQ3_XXS, stock	412	0.2577	0.2577	0.2225
Q2_K, as written	436	0.4900	0.5347	0.3926
Q2_K, re-rounded, wikitext calibration	436	0.3268	0.4255	0.5433
Q2_K, re-rounded, 2:1 wikitext:code calibration	436	0.3690	0.4660	0.2591
Q3_K_M, stock	480	0.1677	0.1678	0.1209

Re-rounded with wikitext calibration against the same file as written: ΔKLD −0.1632 ± 0.0058 on wikitext (t = −28.4, better on 40/40 chunks), 33% lower at identical bytes; the paper's earlier measurement on a pre-fix build was −34% (t = −30.3). On FineWeb-Edu the gain is smaller (−0.1092, t = −12.6). On code the file is worse than the untouched one (+0.1507, t = +13.6).

Against stock rungs: the re-rounded file is ahead of IQ2_M, 44 MB smaller, on wikitext and FineWeb-Edu (t = −25.9 and −15.7) and behind it on code (t = +4.1). The stock IQ3_XXS file, 24 MB smaller, is better than the re-rounded Q2_K on all three corpora (t = +11.7, +22.2, +16.2). A user choosing a file near 430 MB at this size should take

IQ3_XXS. Whether re-rounding would improve IQ3_XXS in turn is not known, because the re-rounder implements only the Q2_K codec.

Calibration decides where the gain lands. We re-rounded the same Q2_K file under five calibration texts, every one disjoint from every evaluation text: wikitext-2 train, a code corpus, a FineWeb-Edu slice taken from documents outside the evaluation slice, a 2:1 wikitext:code interleave and a 1:1:1 interleave of all three. Paired ΔKLD against the file as written, 40 chunks per cell (t in parentheses) [E36]:

calibration	wikitext	FineWeb-Edu	code
wikitext	−0.163 (−28.4)	−0.109 (−12.6)	**+0.151 (+13.6)**
FineWeb-Edu	−0.115 (−16.3)	−0.151 (−16.8)	**+0.088 (+8.9)**
code	**+0.247 (+20.3)**	**+0.275 (+21.1)**	−0.132 (−14.0)
2:1 wikitext:code	−0.121 (−21.4)	−0.069 (−8.2)	−0.133 (−14.6)
1:1:1 wikitext:code:FineWeb-Edu	−0.116 (−20.6)	−0.087 (−10.0)	−0.134 (−14.8)

The pattern is consistent across the grid. Re-rounding transfers between the two prose corpora and does not transfer between prose and code in either direction: every calibration that lacks one of the two kinds of text damages it, by up to half the file's own KLD. Both calibrations that contain both kinds improve all three corpora. On wikitext and FineWeb-Edu they cost against calibrating on that corpus alone (wikitext −0.121 against −0.163); on code they match it (−0.133 and −0.134 against −0.132). The 2:1 interleave was fixed as the deployable recipe before this grid was run. Re-rounding fits the calibration covariance with nothing pulling the codes toward their original values in directions the calibration does not excite, which is a plausible reading of why uncovered text is damaged; we did not test it directly. The effect is not new outside GGUF: a GPTQ re-solve calibrated on one corpus is worse than plain rounding on another at 7B ([23]), and GPTQ is far more sensitive to the calibration language than AWQ ([24]). Upstream tests of llama.cpp's diagonal imatrix statistic found its calibration domain mostly benign (discussions #5006 and #5263; also [25]), so the damage here comes with the full-covariance sweep and not with the imatrix weighting. What the grid adds is the measurement on a deployed K-quant file against the file as written, with code as the uncovered kind.

The calibration sample is a small part of the variance. Re-rounding on three disjoint thirds of wikitext-2 train, the same budget each, moves ΔKLD by at most 0.004 on wikitext, 0.018 on FineWeb-Edu and 0.007 on code (wikitext: −0.163, −0.164, −0.167). This bounds calibration-sampling variance alone at this size, and it is one to two orders of magnitude smaller than the effect of which kind of text is in the calibration set.

Qwen3.6-27B and Llama-3.1-8B (Q2_K; BF16 reference; three corpora; current build). The same sweep at two larger sizes, one of them outside the Qwen family. Every base is one quantization step from BF16 with a 48-chunk wikitext imatrix (at 27B, the imatrix of the earlier capture, made on a build before the normalization fix; at 8B, a fresh one). The re-round arms differ only in the calibration text of their Gram capture, which is fresh, on the current build. 100 chunks per corpus; registered rules [E36]:

file	Qwen3.6-27B: KLD wikitext / FineWeb-Edu / code	Llama-3.1-8B: KLD wikitext / FineWeb-Edu / code
IQ2_M, stock	0.1710 / 0.1314 / 0.1340 (10.30 GB)	0.2891 / 0.2539 / 0.1130 (2.95 GB)
Q2_K, as written	0.1581 / 0.1182 / 0.1188 (11.00 GB)	0.3064 / 0.2701 / 0.1124 (3.18 GB)
Q2_K, re-rounded, wikitext calibration	0.1523 / 0.1359 / 0.1515	0.3011 / 0.3080 / 0.1496
Q2_K, re-rounded, 2:1 wikitext:code calibration	0.1671 / 0.1465 / 0.1130	0.3340 / 0.3299 / 0.1111
IQ3_XXS, stock	0.1077 / 0.0710 / 0.0855 (11.48 GB)	0.1658 / 0.1512 / 0.0620 (3.27 GB)

Paired against the file as written (t):

arm	Qwen3.6-27B: wikitext / FineWeb-Edu / code	Llama-3.1-8B: wikitext / FineWeb-Edu / code
wikitext calibration	−1.87 / **+7.79** / **+10.69**	−1.97 / **+10.22** / **+12.57**
2:1 calibration	**+3.21** / **+13.25** / −1.90	**+8.70** / **+14.17** / −0.66

The registered recipe at size was the mixed calibration that rescued the 0.8B file, and it fails the registered test: it is worse than the untouched file on wikitext and FineWeb-Edu at both sizes and level on code. The wikitext-only

arm is a little better on wikitext (−3.7% and −1.7% in KLD), short of the registered threshold (t = −1.87 and −1.97; the percentile bootstrap intervals, [−0.0124, −0.0002] and [−0.0105, −0.0001], exclude zero and the block-bootstrap intervals do not), and at 27B its NLL on wikitext is worse than the untouched file's (t = +3.71). It damages both held-out corpora at both sizes. The paper's earlier 27B result (−13.3%, t = −7.0) was a legacy pair whose source weights and Grams came from a Q6_K requantization. The same sweep with the earlier capture's BF16 Grams on the single-step base gives −0.0080 on wikitext (t = −2.87), level with the current-Gram arm (t = −1.11), and is worse on both held-out corpora (t = +8.83, +10.29); and the legacy re-rounded file itself, re-scored on the current build, is worse than the single-step file on FineWeb-Edu and code (t = +7.93, +12.53). Even its wikitext lead is local to the first 100 chunks: on the whole test set it is ahead of the single-step file by 0.0043 KLD (t = −3.10), on chunks 101-580 unresolved (t = −0.73), and its wikitext NLL is worse (t = +11.19) [E36]. That the legacy gain came from its Q6_K provenance is an inference: the legacy untouched file no longer exists and was not re-scored, and the two pairs differ in recipe and bytes.

At size the sweep improves its proxy and not the outcome. We dequantized the 27B files with gguf-py's own dequantizer and recomputed the error of three re-rounded tensors (of 256) independently of the sweep's code. Their whitened error under the calibration Gram is 0.49, 0.57 and 0.63 of the untouched file's, which agrees with the ratios of the baseline tool's own gate on the same tensors (§3.4); their unweighted Frobenius error is 16-25% larger; 9-11% of their codes changed [E36]. The headroom was there: a file that stores every tensor the sweep covers at Q8_0 instead of Q2_K has KLD 0.034 / 0.023 / 0.029, so those tensors carry 76-80% of the Q2_K file's divergence, and the sweep removed about 5% of it on wikitext. This is the largest surrogate-against-outcome gap in the paper (§4.7). One reading fits the held-out damage: the sweep moves error out of the directions the calibration activations occupy and into the others. We did not test it directly (the test is the same computation under a Gram captured on held-out text), it rests on three tensors, and it does not explain why the gain on wikitext itself is small. Concurrent work reports the same reversal for frozen-grid code refinement in generic group-wise quantization, where code changes that lower the calibration reconstruction loss raise held-out perplexity in 47-74% of single-layer replacements, gives this mechanism for it, and repairs it with a worst-case objective over activation distributions that improves held-out text at 8B and 70B (DRQ, [8]); we did not test that objective on K-quant grids. The quantity that rose here, the unweighted error, is the one a linearity result ties to perplexity for near-isotropic error ([26]); in these three tensors it moved with the outcome, and the calibration-whitened proxy did not.

What separates 0.8B from 8B and 27B is not established by this design. The same split, reconstruction that transfers at a small size and overfits at a larger one, was reported for pruning by reconstruction ([27]). Candidates we did not test: the amount of calibration text per input dimension (20 calibration tokens per input dimension at 0.8B, against 6 at d = 4,096 and 4.8 at d = 5,120, so the larger Grams are estimated far less precisely); the share of the model the Q2_K codec reaches; and the larger KLD of the 0.8B Q2_K file, which leaves more to remove.

The negative result at size is for this sweep: one-shot, layer-local, on an unrotated grid. Other re-selections inside the same formats report gains at 8B. GSQ keeps the Q3_K scales frozen and trains the codes against block outputs, with gains at 4B and 8B ([9]); a gradient-tuned re-selection inside q2_k_s gains on Llama-3.1-8B against its own optimized rounding (AutoRound, [28]); and a non-archival equal-bytes study puts a Hadamard-rotated, LDLQ-rounded file at IQ2_XXS size well ahead of IQ2_XXS on Llama-3.1-8B, most of the gain coming from the rotation and the rounding (Zenodo DOI 10.5281/zenodo.23481757). We ran none of them.

What the tool requires. The re-rounder is not a transformation of a GGUF alone. It needs the higher-precision source weights, a Gram capture made with our patched llama-imatrix, calibration text that covers every kind of text the file will serve, and a held-out check. Its output is a standard GGUF.

3.2 A richer metric made the result worse

The natural extension of input-only re-rounding is the two-sided Kronecker metric $\mathrm{tr}(T\,\Delta W\,S\,\Delta W^{\mathrm{T}})$, with S the input Gram and T the output-gradient Gram: the Kronecker-factored Hessian of YAQA ([29]) with our captured factors. The nested sweep of §2.3 reduces that surrogate (about 35% on correlated synthetic problems) and reproduces the input-only sweep exactly when T = I. End to end [E24]:

comparison (0.8B, paired)	ΔNLL (n = 580)	ΔKLD (n = 40)	Δtop-1 (n = 40)
two-sided vs as written	−0.1154 ± 0.0042 (t = −27.7)	−0.1193 ± 0.0081	+5.58 ± 0.50
two-sided vs input-only	+0.0483 ± 0.0031 (t = +15.5)	+0.0475 ± 0.0058 (t = +8.2)	−0.69 ± 0.42

The two-sided sweep with our measured T gives back 29% of the input-only gain (t = +15.5 on NLL). The solver does what the mathematics says; the estimate of T is the weak part. It comes from a single 16,384-token pass, carries the gradient cut of §2.1 on this architecture, and its late-layer eigenstructure is at the fp32 noise floor by the capture tool's own checks.

A pre-registered run on Qwen3-0.6B, which has no gradient cut, asked whether the cut was the cause. There the two-sided sweep is worse than the input-only sweep and worse than the untouched file: perplexity 50.53, KLD 0.866, top-1 58.0%, against 35.54 / 0.556 / 64.0% input-only and 43.33 / 0.766 / 58.2% as written. The model changed along with the capture, so magnitudes are not comparable, but the direction rules out capture coverage as the explanation. These 0.6B numbers are recorded in the experiment log only; the evaluation log was not kept [journal].

A second surrogate result points the same way: alternating the code sweep with a refit of the storable scales lowers the whitened objective a further 2.6% and changes no end-to-end metric (Appendix A.4). §4.7 collects every such case.

3.3 Ordering: re-round a bare file, alternate when a corrector will be attached

Re-rounding helps a bare file and hurts under a corrector. At 0.8B, the bare base with a freshly extracted full-covariance correction beats the re-rounded base with one (ΔNLL t = +17.6, n = 580; the re-rounded stack is better on 133 chunks). The same ordering was seen at 0.6B, unpaired: re-round then correct 26.80 perplexity, a one-shot re-round under a ProjQ-style modified metric ([30]) 25.94, alternation then correct 25.27 [E24, E13]. Re-rounding minimizes the standalone whitened error and spends grid resolution on directions the corrector would have absorbed. The guidance, for the small models where re-rounding helps at all (§3.1): re-round a bare file only with a calibration that covers its intended text, alternate (§2.4) when a corrector will be attached, and do not stack re-rounding under a corrector.

3.4 Freezing the grid against refitting it

A referee asked whether keeping llama-quantize's grid is a constraint worth its cost, against GPTQ allowed to choose the grid as well. We built that baseline at identical format and bytes: the same sweep, Grams, damping and tensor coverage as §2.3, except that when the sweep enters each 256-column superblock it refits that superblock's grid from the error-updated weights with llama.cpp's own Q2_K quantizer and the same imatrix. Its grid fit reproduces llama-quantize's bytes exactly (all 96 covered tensors at 0.8B and all 256 at 27B, refitting the original weights), and with error feedback switched off the whole tool reproduces llama-quantize's file byte for byte at 0.8B and on the five 27B tensors we ran that way, so the only difference between the two arms is whether the grid is frozen. On the whitened objective the refit grid is lower than the frozen one on all 96 tensors at 0.8B (by 12% in total) [E36].

Paired ΔKLD, free-grid minus frozen-grid at identical bytes, same Grams (t) [E36]:

model, calibration	wikitext	FineWeb-Edu	code
Qwen3.5-0.8B, wikitext	−0.017 (−4.53)	−0.028 (−5.49)	−0.058 (−7.33)
Qwen3.5-0.8B, 2:1	−0.017 (−3.63)	−0.021 (−3.62)	−0.023 (−7.47)
Llama-3.1-8B, 2:1	−0.018 (−6.43)	−0.010 (−3.08)	−0.007 (−3.62)
Qwen3.6-27B, 2:1	−0.0095 (−3.22)	−0.0059 (−2.64)	−0.0046 (−1.83)

Refitting the grid is better than freezing it in eleven of twelve cells and level in the twelfth. Freezing llama-quantize's grid is therefore a restriction that costs quality and not a feature of the method, and the method is better described as GPTQ inside the GGUF formats, with the frozen grid as its zero-search special case. This confirms inside GGUF what has been shown outside it: refitting the grid to the codes beats a scale refit alone in all 16 comparisons of JARQ ([31]), and SchurQuant's fixed-against-refit ablation points the same way. GPTQ with the K-quant grid refit at each superblock has also been released as software (IST-DASLab gptq-gguf-toolkit, 2025); we did not run it. The refit does not change the verdict at size: at 8B and 27B the free-grid file is still worse than the untouched file

on FineWeb-Edu (t = +8.29, +8.16) and not better on wikitext (+2.87, −0.19); it is better on code under the code-containing calibration (−3.85, −4.08).

This baseline does not cover learned rounding (AdaRound and its successors), which needs a block-reconstruction training loop, or quantizers that add incoherence processing, trained codebooks or trellis codes; §7.1 lists both.

4. Results II: where low-rank correction falls short

This section reports the correction audit. Most of its results are negative, and several of them are the best-resolved findings of the study [review].

4.1 Unweighted spectra: neither the weights nor the residual are low-rank

A singular-value census over all 197 tensors of Qwen3-0.6B finds that capturing 95% of Frobenius energy needs 50-80% of full rank on every projection type. At that rank a factorization saves nothing (best case attn_q at 1.09×; value and FFN projections cost memory) [E1]. The quantization residual is flatter: 90% of its energy needs about 62% of full rank, as the Marchenko-Pastur analysis of LQER ([2]) predicts and as RILQ ([6]) reports for 2-bit quantization. An unweighted rank-128 F16 correction of a Q2_K base recovers 26-44% of error energy at a cost of 5.26 bits per weight, more than Q4_K [E3].

This statement is scoped to the unweighted Frobenius norm on one 0.6B model. Under activation whitening the picture differs: at 27B, rank 128 captures 76% of the whitened weight matrix on a 12-tensor sample (§4.4). That concentration does not convert into quality at equal bytes, which is the subject of the rest of this section.

4.2 Capture against rank and width

Diagonal-whitened rank-64 capture of the Q2_K residual is 0.37 at width d = 1,024 (0.6B) and 0.07 at d = 5,120 (27B). To replace that two-point reading we recomputed whitened spectra for 1,146 tensors at 10 ranks under 3 whiteners on four models: Qwen3-0.6B and Qwen3.5-0.8B at d = 1,024, and Qwen3.6-27B and Qwen3.8-27B at d = 5,120. We fitted capture $\approx a \cdot r^{b_r} \cdot d^{b_d}$ [E33].

fit	whitener	b_r	b_d
all four models as shipped or built	diagonal	0.61 [0.60, 0.61]	−0.67 [−0.70, −0.64]
all four models as shipped or built	full Gram	0.44 [0.44, 0.45]	−0.59 [−0.61, −0.57]
one quantization recipe on every model (primary)	diagonal	0.68 [0.67, 0.68]	−0.76 [−0.79, −0.73]
one quantization recipe on every model (primary)	full Gram	0.37 [0.36, 0.37]	−0.52 [−0.54, −0.50]

Capture is sublinear in rank and, across a single width step, falls more slowly than r/d predicts: the linear form capture = c·(r/d) is the worst of six candidate forms under every whitener, and with the recipe fixed, holding out either 27B model predicts it to within 0.9-1.1×. Whitening by the full Gram reduces the width exponent by about a third (−0.76 to −0.52). The prefactor depends on the base recipe and not on the model: requantizing Qwen3.8-27B from BF16 with the recipe used for Qwen3.6-27B makes the two models indistinguishable (0 of 40 per-kind contrasts exclude zero), while Unsloth's shipped dynamic quantization of the same model sits on a different curve.

Scope: the width axis is one step (1,024 ⟶ 5,120) and is confounded with model size, depth and family. The two 27B models share a width. The intervals resample tensors (1,000 cluster bootstraps), not models, and the fit treats ranks and whiteners of one tensor as separate points. A "constant outlier head plus isotropic bulk" form fits a head fraction of zero, so the outlier-head explanation ([32], [33], [34]) is not supported by this regression. A third width is required before any exponent is quoted as more than a fit [rollup, review]. What is established is the direction: at these widths a deployable rank is a small fraction of dimension, and fixed-rank capture falls as the model widens, consistent with the bounds of CALDERA ([5]) and [20].

4.3 Correction against the stock types, on and off the calibration distribution

27B: files and protocol. Every 27B file in this subsection is one quantization step from the BF16 reference, with a 48-chunk BF16 imatrix and Gram capture on wikitext-2 train [E27]. These files were built in the earlier campaign, so their imatrix and the correction's Grams were captured on a llama.cpp build before the normalization fix; only the scoring is on the current build. The corrected configuration is the mixed base (FFN tensors at Q3_K, the rest

at Q2_K) plus a rank-128 full-covariance correction with Q8_0 factors on the non-FFN tensors (336.5 MB; mean whitened capture 0.42). The control is a mixed-type file with no adapter, built once at the corrected configuration's byte count (Q3_K_S with attn_k, attn_v and attn_output promoted to Q4_K and a partial attn_qkv promotion; −0.016% bytes), its bytes predicted in advance from measured per-kind sensitivities. All rows below were scored on llama.cpp b10865 against BF16 reference logits generated on that build, on 100 chunks of three corpora: wikitext-2 test (the calibration distribution), FineWeb-Edu and a code corpus disjoint from the code used anywhere in calibration (§2.5). The comparison and its decision rules were registered before the run [E36].

file	GB	KLD, wikitext	KLD, FineWeb-Edu	KLD, code
mixed base, no adapter	12.16	0.0971 ± 0.0025	0.0731 ± 0.0014	0.0830 ± 0.0021
IQ3_XS	12.26	0.0766 ± 0.0022	0.0517 ± 0.0010	0.0632 ± 0.0016
Q3_K_S	12.37	0.0910 ± 0.0022	0.0638 ± 0.0012	0.0746 ± 0.0020
corrected configuration	12.50	0.0863 ± 0.0023	0.0686 ± 0.0014	0.0779 ± 0.0020
mixed-type control	12.50	0.0813 ± 0.0022	0.0577 ± 0.0011	0.0666 ± 0.0018
Q3_K_M	13.59	0.0605 ± 0.0016	0.0410 ± 0.0008	0.0500 ± 0.0015

Paired per-chunk KLD differences, n = 100 per corpus (t; A ahead on how many chunks):

pair (A − B)	wikitext	FineWeb-Edu	code
corrected vs Q3_K_S	−0.0047, t = −2.62 (71/100)	+0.0048, t = +3.26 (40/100)	+0.0032, t = +1.86 (39/100)
corrected vs control	+0.0050, t = +2.37 (40/100)	+0.0108, t = +6.48 (10/100)	+0.0113, t = +7.11 (20/100)
control vs Q3_K_S	−0.0097, t = −8.64 (91/100)	−0.0060, t = −13.37 (98/100)	−0.0080, t = −6.90 (81/100)
corrected vs mixed base	−0.0108, t = −10.41 (96/100)	−0.0045, t = −8.63 (91/100)	−0.0051, t = −6.48 (81/100)
mixed base vs Q3_K_S	+0.0061, t = +2.93 (31/100)	+0.0093, t = +5.70 (23/100)	+0.0083, t = +4.32 (30/100)
IQ3_XS vs corrected	−0.0097, t = −3.58 (83/100)	−0.0169, t = −8.55 (97/100)	−0.0147, t = −7.06 (81/100)

On the calibration distribution the corrected configuration is ahead of Q3_K_S by 0.0047 KLD (5%; t = −2.62; Wilcoxon p = 2e-5; block bootstrap over contiguous blocks of 10 chunks [−0.0081, −0.0009]) and behind the same-byte control by 0.0050 (t = +2.37; Wilcoxon p = 0.0014; block bootstrap [+0.0017, +0.0091]). In NLL it is ahead of Q3_K_S (t = −5.79) and level with the control (t = +0.21). With a Bonferroni factor of six for the pairs in the table, the two contested KLD margins have p = 0.061 and 0.12; the registration promised these adjusted values beside each contested pair, and we report them here. This is a reproduction and not an independent sample: the same files and the same 100 chunks were first scored on a build that predates llama.cpp's Gated-DeltaNet normalization fix (b10829), where the two margins were −0.0048 (t = −2.85) and +0.0063 (t = +2.62); the per-chunk differences of the two runs correlate at 0.87 for this pair (0.95 for the pair with the control). The margin over Q3_K_S is not uniform across these 100 chunks on either build: on chunks 1-40 it is +0.0004 (t = +0.13) and on chunks 41-100 −0.0081 (t = −3.99). The correction's increment over its base is the same in both blocks (−0.0106 and −0.0109); what changes is the base, which is behind Q3_K_S by 0.0110 on chunks 1-40 (t = +3.38) and by 0.0028 on chunks 41-100 (t = +1.07). That pre-fix 100-chunk sample was reached through looks at 20 and 40 chunks (each a prefix of the next) with a registered rule for parity and none for a win [E35, E36].

The whole wikitext test set. Because the 100-chunk sample was reached through looks and its margin over Q3_K_S sits in 60 of its chunks, we scored all nine 27B files on every 512-token chunk of the wikitext-2 test set (580 chunks, a fresh BF16 reference on the same build), with the decision rules registered before the run; the first 100 chunks reproduce the rows above bit for bit. Both wikitext margins hold and are resolved. The corrected configuration is ahead of Q3_K_S by 0.0042 KLD (4.5%; t = −4.32; ahead on 363/580 chunks; block bootstrap [−0.0062, −0.0022]; Bonferroni-adjusted p = 1.1e-4) and behind the control by 0.0051 (t = +5.50; control ahead on 371/580; adjusted p = 3.4e-7). On chunks 101-580 alone, which share no chunk with the tables above, the two margins are −0.0041 (t = −3.68) and +0.0051 (t = +4.96). The other pairs keep their direction: IQ3_XS ahead of the corrected configuration by 0.0106 (t = −9.06), the correction's increment over its base −0.0108 (t = −19.8), the base behind Q3_K_S by 0.0066 (t = +7.08). In NLL the corrected configuration is ahead of Q3_K_S (t = −10.97) and of IQ3_XS (t = −5.54), and the control is ahead of it (t = −2.26), where on 100 chunks the two were level [E36].

Off the calibration distribution the margin over Q3_K_S does not hold. On FineWeb-Edu it reverses: Q3_K_S is ahead of the corrected configuration by 0.0048 (t = +3.26, block bootstrap [+0.0024, +0.0073]). On code it is

unresolved and leans the same way (+0.0032, t = +1.86). The control's lead over the corrected configuration roughly doubles off-distribution (t = +6.48 and +7.11; ahead on 90/100 and 80/100 chunks), and IQ3_XS, which is smaller and needs no adapter, is ahead of every file in the group except Q3_K_M (1.1 GB larger) on all three corpora; in wikitext NLL, though, the corrected configuration is ahead of IQ3_XS (t = −2.42). The reversal decomposes into its two parts. The mixed base is further behind Q3_K_S off-distribution (+0.0093 and +0.0083, against +0.0061), and the correction, whitened by wikitext Grams, recovers less than half as much there (−0.0045 and −0.0051, against −0.0108). It still improves its base on every corpus.

Reading. The narrow 27B advantage of the corrected configuration over the adjacent stock type is real on the calibration distribution, where the whole test set resolves it, and does not carry off it. On the first 40 wikitext chunks it is absent; across the other 540 it holds. A stock mixed-type file at the same bytes is better on every corpus we measured, and IQ3_XS is better at fewer bytes. What survives off-distribution is the correction's increment over its own base, at about half its on-distribution size; §4.5 measures the same halving along a recovery curve.

These verdicts are for symmetric correction, the class of QERA's closed form that ours belongs to. A two-stage compensator with an asymmetric, output-Fisher-weighted target recovers 51% and 84% of the held-out C4 gap on a 2-bit QuIP# base at Qwen3-8B and 4B, against 31% and 63% for the best baseline it compares, and argues that the symmetric target is itself high-rank ([12]). It builds no byte-matched control, and we did not run it against ours, so whether the stronger target closes the distance to the same-byte control is open. The same-byte question was first posed upstream: a 4.30-bpw L²QER result trailed IQ4_XS at 4.25 bpw on four LLaMA models, and an informal test on a 270M model found a carve plus correction no better than the stock Q2_K mix (ik_llama.cpp discussion #15). Allocating llama.cpp K-quant types per tensor at near-equal bytes beats the stock IQ3_M mix on Qwen3.8-27B as well ([35]). What this subsection adds is the controlled version: paired, scored off the calibration distribution, at 0.8B, 8B and 27B.

A task-level check, run before the held-out pass, does not separate the pair. On the HellaSwag validation set (10,042 items, identical per file) the corrected configuration, Q3_K_S and Q3_K_M score 82.25%, 82.39% and 82.85%; no gap reaches the pre-registered 1.4-point threshold. A post-hoc paired McNemar test finds the corrected configuration and Q3_K_S tied (z = −0.81, with the sign opposite to the wikitext KLD difference) and Q3_K_M ahead of both (z = +3.48 and +2.82) at a KLD difference of about 0.03 [journal].

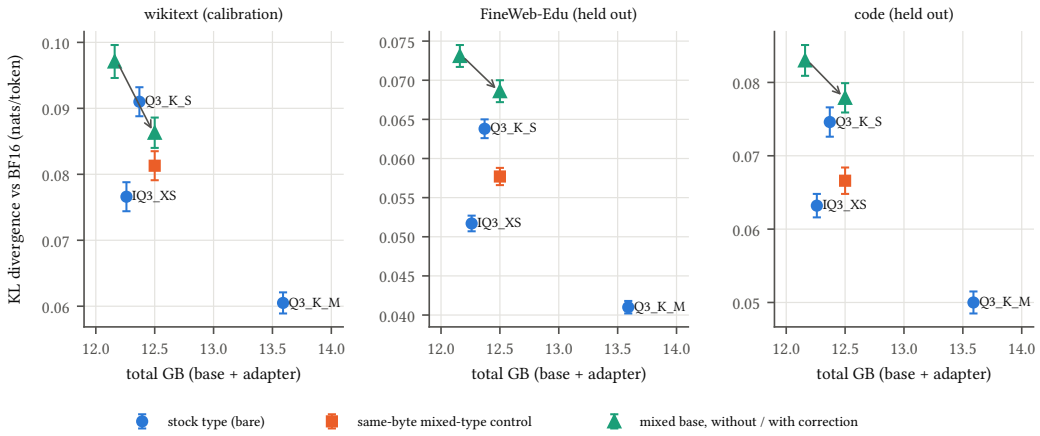


Figure 1: Qwen3.6-27B near 12.5 GB on three corpora, 100 chunks each, every file one quantization step from BF16, scored on llama.cpp b10865; error bars are standard errors. On wikitext, the calibration distribution, the correction (arrow) moves the mixed base past the stock type Q3_K_S; on FineWeb-Edu and code it does not. A mixed-type control with no adapter at the same bytes is lower on every corpus, and IQ3_XS is lower still at fewer bytes; only Q3_K_M, 1.1 GB larger, is below it.

Llama-3.1-8B: outside the Qwen family. The same protocol on Llama-3.1-8B base (the plain transformer used across the quantization literature; d = 4,096): bases one step from BF16, a 48-chunk wikitext imatrix and Gram capture, the mixed base plus a rank-128 Q8_0 correction on the attention tensors (115.9 MB), and a control by the same promotion rule. Here the corrected configuration is only 12.2 MB above Q3_K_S, so the rule stopped inside its first step (attn_k at 21 of 32 layers); this control is Q3_K_S plus 11.7 MB, which was stated before the run [E36].

file (Llama-3.1-8B)	GB	KLD, wikitext	KLD, FineWeb-Edu	KLD, code
IQ3_XS	3.52	0.1240	0.1155	0.0435
mixed base, no adapter	3.56	0.1919	0.1738	0.0726
Q3_K_S	3.66	0.1914	0.1667	0.0653
control	3.68	0.1884	0.1635	0.0639
corrected configuration	3.68	0.1698	0.1585	0.0670
Q3_K_M	4.02	0.0780	0.0724	0.0235

On this model the correction does better than on Qwen. It is ahead of Q3_K_S on wikitext (−0.0216, t = −17.40) and on FineWeb-Edu (−0.0082, t = −3.81), and level on code (t = +1.44); against the near-Q3_K_S control it is ahead on the two prose corpora (t = −14.27, −2.18) and behind on code (t = +2.49). Its increment over its own base is resolved on all three (t = −19.3, −11.9, −11.0). IQ3_XS, 4% smaller and with no adapter, is ahead of it on every corpus (t = −18.7, −14.9, −12.5) and by more than the correction's whole increment. On Llama, then, a calibrated correction beats the adjacent K-quant type on FineWeb-Edu as well as on wikitext and is level with it on code, and the I-quant type below it in bytes beats both on every corpus.

0.8B: the control at small scale. At 0.8B the paper's earlier comparison chose its rung from below and built no control. We built one by a rule fixed in advance: start from Q3_K_M and promote whole tensor kinds to Q4_K in the order of the 27B sensitivities until the file reaches the corrected configuration's bytes from below. At 0.8B, Q3_K_M already holds four of the listed kinds at Q4_K or wider, so the list ran out at 517.5 MB, 1.1% below the 523.3 MB corrected configuration (Q2_K with one alternation round and a rank-128 full-covariance correction with Q8_0 factors). A second control matched to the byte (523.0 MB, a further pass at Q5_K) was designed after the registration and is reported as such. Both were built once and scored on 40 chunks of the three corpora against BF16 references on the current build [E36]:

file (Qwen3.5-0.8B)	MB	KLD, wikitext	KLD, FineWeb-Edu	KLD, code
Q2_K	436	0.4900	0.5347	0.3926
IQ3_XXS	412	0.2577	0.2577	0.2225
Q3_K_M	480	0.1677	0.1678	0.1209
corrected configuration	523	0.1474	0.1966	0.1805
control (registered rule)	517	0.0489	0.0470	0.0338
control (byte-matched, post-registration)	523	0.0432	0.0426	0.0285
Q4_K_M	543	0.0397	0.0380	0.0279

The registered control is ahead of the corrected configuration by 0.099 KLD on wikitext (t = −26.7), 0.150 on FineWeb-Edu (t = −32.0) and 0.147 on code (t = −21.0), on every chunk of every corpus, with fewer bytes; its KLD is a third of the corrected configuration's. The corrected configuration is ahead of Q3_K_M on wikitext (t = −5.75) and behind it on FineWeb-Edu (t = +7.77) and code (t = +14.48). At this size the corrected file is far below the line that stock types trace when they are mixed at its bytes. The control is a single point built by a sensitivity-ordered rule, not an optimized allocation: a systematic allocator would likely widen the gap, and a badly guided reallocation can lose even to the uniform file ([36]). At 0.6B (legacy rows, scored against Q8_0) the corrected configuration was compared only with Q3_K_M below it, and linear interpolation between the neighbouring rungs was already better; no control was built there [rollup, review].

A correction on an NVFP4 base at 0.8B improves its base and loses to every stock rung at or below its bytes; the details are in Appendix A.1.

A qualification on all of the above. Every corrected configuration pays a decode cost of 20-40% on stock llama.cpp that the bare files do not (§6).

4.4 Three placements of the low-rank bytes, and a bound on their difference

A set of pre-registered paired experiments at 27B asked whether it matters where the low-rank bytes act. Three placements were built at matched bytes on the mixed base. The first is the post-hoc correction of §2.2. The second is a pre-quantization carve in the preserve-then-quantize order ([37]): take a whitened rank-128 component C = B·A off the BF16 weights, quantize the remainder with the same mixed recipe, and serve C as the adapter. The

third shares one A factor across tensors that read the same input (the arrangement of [38]), in two variants: at byte parity with the first placement, and at rank 128.

Their surrogate scores differ. The post-hoc correction captures 0.42 of the whitened residual. The carve captures 76% of the whitened weight matrix at rank 128 on a 12-tensor sample. The shared basis at byte parity captures 12.2% more whitened energy than separate bases on the attention groups and 6.9% more on the GDN groups; at equal rank it keeps 99.55% and 99.75% of the separate capture [E28, E29].

Their measured quality does not. Each alternative placement minus the post-hoc correction, on 100 paired chunks, all scored on a build that predates the normalization fix (§2.5) and paired only with each other; the correction's increment on that build is 0.0090, against 0.0108 for the same files on b10865 (§4.3) [E35]:

placement vs post-hoc correction	ΔKLD	t	90% interval	largest bound as a share of the correction's increment (0.0090)
pre-quantization carve	+0.0017 ± 0.0017	+0.99	[−0.0011, +0.0045]	49%
shared basis, byte parity	−0.0005 ± 0.0006	−0.91	[−0.0015, +0.0004]	17%
shared basis, rank 128	−0.0008 ± 0.0006	−1.48	[−0.0018, +0.0001]	20%

Three low-rank placements differ by less than 0.0045 KLD (90% interval), under 50% of the correction's own increment (0.0090); for the two shared-basis variants the bound is 0.0018, about 20%. We could not distinguish them. This is a bound and not a demonstration that the placements are the same: the interval for the carve admits a deficit of half an increment, its top-1 difference leans negative (t = −1.68), and it trails the control by more than the others do. One mixed-type control at the same bytes is better than all four files (t = +2.62, +5.15, +2.35 and +2.39 for the post-hoc correction, the carve, and the two shared-basis variants).

Scope: one model, one byte count, one rank, one corpus, with three arms that share a base recipe and a Gram. The statement that type allocation does better rests on one designed control. Per-tensor type allocation is the axis that RCO ([39]) optimizes under an exact byte budget; that paper does not compare placements, and this one does not optimize allocation.

A related family uses seeded, unlearned bases (AWSRC, [40]). A pre-registered capture comparison on our cached whitened spectra (18 tensors, 3 seeds) finds that such bases capture 0.035 of the dominant whitened energy at byte parity against 0.983 for learned factors, which is the random-subspace expectation [E31]. We did not run a quality comparison for that family.

One by-product: the shared basis at rank 128 is within the bound above of the post-hoc correction while storing fewer distinct factor bytes. If a loader aliased the shared factor, the adapter would occupy 280.8 MB against 336.5 MB (−16.6%). That figure is a computed size under a loader patch we did not write; the file that was evaluated stores the duplicates and is 336.5 MB [E28].

4.5 Recovery grows as the base gets thinner, and factors do not replace base bits

We held the correction recipe fixed and varied only the base: the same rank-128 full-covariance correction with Q8_0 factors on the non-FFN tensors, whitened by one wikitext Gram capture, extracted on three 27B bases each one quantization step from BF16, and scored on 100 chunks of wikitext and of FineWeb-Edu on the current build. Recovery is the fraction of the base's KLD that the correction removes, with a 95% paired bootstrap interval [E36]:

base	GB	KLD wikitext, base → corrected	recovery, wikitext	KLD FineWeb-Edu, base → corrected	recovery, FineWeb-Edu
Q2_K	11.00	0.1581 → 0.1470	7.0% [5.2, 8.8]	0.1182 → 0.1143	3.3% [2.5, 4.1]
IQ2_XS	9.38	0.2507 → 0.2201	12.2% [10.2, 14.4]	0.1875 → 0.1732	7.6% [6.8, 8.5]
IQ1_S	7.44	0.9348 → 0.6356	32.0% [30.1, 33.9]	0.6588 → 0.5513	16.3% [15.3, 17.2]

Every increment is resolved (t from −6.3 to −29.1; the corrected file is ahead on at least 86 of 100 chunks). Recovery rises monotonically as the base gets thinner, which agrees with the published claim that correction pays most at the lowest bit widths ([41]) and with EoRA's results. On FineWeb-Edu it is 0.47-0.62 of the wikitext value at the three bases, close to the ratio of the correction increments in §4.3. This curve replaces an earlier one (20%, 28%, 41%, legacy rows) whose points differed in adapter budget and rank mix as well as base.

Recovery is not the same as competitiveness. The corrected IQ1_S file (7.78 GB with its 0.34 GB adapter) has KLD 0.636 on wikitext, against 0.251 for the bare IQ2_XS file at 9.38 GB. Pushing the idea further, IQ1_S with 2.52 GB of factors still loses to bare Q2_K at near-equal total bytes by about a factor of three in KLD; per gigabyte, added factor bytes bought about 0.06 KLD where base bits bought 0.22 to 0.32 (legacy rows, Appendix A.2).

4.6 Mixture-of-experts: correction loses to a one-line promotion (n = 20, Q4-referenced)

On Qwen3.6-35B-A3B (256 experts per stack, 8 routed; expert stacks are 91% of parameters) both correction variants improve their base and both lose to the ladder, with a control built from the start; every file in this subsection was scored on a build that predates the normalization fix (§2.5) [E23]. Two limits apply to every number in this subsection: n = 20 chunks, and the reference is not BF16. We had no BF16 of this model, so every file is one quantization step from the vendor UD-Q4_K_M file (22.1 GB) and KLD is measured against that file's logits.

Per-expert whitened correction (rank 32, Q8_0 factors, pooled-Gram whitening; served on stock llama.cpp, which handles three-dimensional adapter pairs) improves the Q2_K base (t = −4.55, 18/20 chunks); a shared-basis variant improves it weakly (t = −2.69, 13/20). At its byte count (14.5 GB) the per-expert correction loses to a control that promotes ffn_down_exps to Q4_K, by +0.0257 ± 0.0038 KLD (t = +6.85, 0/20 chunks), and the shared-basis variant loses to the same control at t = +9.33. Q3_K_S, 0.7 GB larger than the corrected file, is lower again (KLD 0.1150 against the control's 0.1666; control versus Q3_K_S, t = +8.15). Per byte added to the base, the promotion control removes 2.6 times as much KLD as the rank-32 factors, and Q3_K_S 4.2 times as much [E35].

Because the control promotes a tensor group toward the reference's own bit width, we checked whether the reference favours it by construction. In the vendor reference, ffn_down_exps is Q5_K in 37 layers and Q6_K in 3 (from the quantize log and from the GGUF header); the up and gate stacks are Q4_K. The control's Q4_K is therefore a real quantization step below the reference and not a copy of it. The comparison still measures distance to a quantized model and should be repeated against BF16 when one is available [E35].

The structural result is about sharing. One output basis per stack, shared by 256 experts, captures 0.086-0.090 of whitened residual energy on the up and gate stacks against 0.0625 for a random orthonormal basis, and 0.022 against 0.0156 on the down stacks. The pooled whitened residual is nearly isotropic, so there is no common subspace of the residual to amortize at rank 32. This is a statement about the quantization residual: shared bases across experts do exist for the weights themselves, which several mixture-of-experts compressors exploit (e.g. [42]). Each expert is a 512 × 2,048 matrix whose residual is weakly structured (per-expert capture 0.19-0.36, measured under the pooled whitener, which approximates each expert's own input covariance), and the factor overhead is paid 256 times. Correcting the dense tensors of a mixture-of-experts model, or expert bases below 2 bits where no cheap promotion exists, were not tried.

4.7 When a better surrogate did not give a better outcome

Metric quality mattered more than rank. Going from diagonal to full-covariance whitening at 0.6B was worth more than doubling rank: full covariance at rank 64 gives perplexity 27.58 and KLD 0.318, against 31.29 and 0.420 for the diagonal at rank 64 and 28.46 for the diagonal at rank 128 (about 10 and 2.6 unpaired standard errors). The same comparison at 27B was about 2 unpaired standard errors (0.1460 ⟶ 0.1282 at rank 64) [E10].

In the other direction, improving a surrogate repeatedly failed to improve the outcome.

Rank allocation by greedy energy water-filling at 0.6B is worse than uniform rank at equal bytes (perplexity 34.17 against 31.29); it assigns no rank to ffn_down. Measured per-kind KL sensitivities give a different ordering (attn_k highest at 0.0128 ΔKLD per MB; isolated sensitivities sum to 0.352 against a joint 0.346). The end-to-end margin of allocated over uniform rank at 27B is unresolved (0.5 unpaired standard errors), and the 27B weights were priors carried over from 0.6B [journal, review].

Sensitivity estimated from one side anti-correlates with measured sensitivity (Spearman −0.29 for the gradient side, as for the input side); the two-sided product correlates positively (+0.39) [E20]. Measured two-sided full-covariance correction at 0.6B gives 27.27 against 27.58 one-sided at equal bytes.

The full list, with the cases reported in other sections:

#	what improved on the surrogate	what the outcome did	where
1	energy captured by greedy rank allocation	perplexity worse than uniform (34.17 vs 31.29)	this section
2	exact descent of a two-sided Kronecker metric with measured T	+0.048 nats vs input-only (t = +15.5); worse than the untouched file with a cleaner T	§3.2
3	a further 2.6% descent of the same whitened objective	no change ($\|t\| \le 1.32$)	Appendix A.4
4	importance moved into column scales (equalization)	KLD worse at every strength tested (α = 0.25 to 1; t = +5 to +29 on wikitext, same ordering on FineWeb-Edu and code), with or without an imatrix	Appendix A.3
5	whitened capture at equal bytes (shared basis, +12% on attention groups)	KLD within the bound of §4.4 (t = −0.91)	§4.4
6	residual capture of the base (NVFP4 0.736 vs Q3_K_M 0.721)	smaller gain (50% vs 70% recovery)	Appendix A.1
7	KL divergence to the reference, across refinement families	opposite ordering to NLL in the same run at 2,048 tokens, and to a coding suite	§5.4, §5.5
8	whitened error of the swept tensors halved at 27B (Frobenius error +16-25%)	KLD −3.7% on wikitext, unresolved; worse on both held-out corpora; 5% of the available headroom	§3.1

Rows 1-6 and 8 are surrogates for KL divergence that did not predict KL divergence. Row 7 is of a different kind: KL divergence measures distance to the reference model, and a refined file can move away from the reference while assigning the text as high a likelihood as the reference itself does (§5.4). Within one refinement family we have no evidence that KL divergence predicts task outcomes at the size of the contested margins either: the one task evaluation we ran there (HellaSwag, §4.3) tied the contested pair.

4.8 Conditioning the correction on a task distribution

Conditioning the correction's covariance on a code corpus gives 10.3% lower perplexity on code at identical bytes (2.746 against 3.061 for the wikitext-conditioned correction; about 8 unpaired standard errors) and costs 27% on wikitext (34.90 against 27.58). Broad calibration transfers to a narrow distribution better than the reverse [E17]. Per-workload adapters are possible through llama.cpp's per-request adapter API.

The calibration grid of §3.1 measures the same question at 0.8B on KL divergence: whichever of five calibration texts the correction is whitened with (wikitext, code, FineWeb-Edu, or a 2:1 or 1:1:1 interleave), it lowers its base on all three evaluation corpora ($|t| \ge 19$ in every cell), with the largest gain on the calibrated distribution. Unlike re-rounding (§3.1), the correction does not damage text it was not calibrated on; it helps it less. The measurement is also a caution: the method's sensitivity to distribution (10-27% here; a halving of the increment at 27B, §4.3 and §4.5) is larger than every contested margin, which is why every comparison in §3-§5 now carries held-out corpora.

4.9 Ordering of operations

From §3.3 and the alternation runs: shaping the base and the corrector jointly beats composing them greedily. At 0.6B, two alternation rounds (perplexity 25.27) are better than a one-shot re-round under a ProjQ-style metric (25.94), which is better than re-round-then-correct (26.80); these are unpaired, and the ordering is paired-resolved at 0.8B (§3.3) [E13, review]. The alternation is LoftQ's; what is specific here is the black-box llama-quantize grid and the whitened metric. A second rule comes from the mixed base and from §4.5 and §4.6: put the correction where rank is a large fraction of width, and spend base bits elsewhere.

5. A head-to-head with released trained checkpoints

5.1 Setup, bytes and cost

GSQ-RCO ([9], [39]) is released as refined GGUF checkpoints of Qwen3.8-27B. By its model card, every tensor is quantized with GSQ (Gumbel-Softmax re-selection of codes) at every candidate GGUF type, RCO selects one type per tensor under an exact byte budget, and the selections are assembled into a standard GGUF. The public GSQ repository contains no GGUF path, so we compare the released files and make no statement about the mechanism that produced them. The comparison was pre-registered on the Unsloth Dynamic baselines the card compares

against, scored on 40 chunks of wikitext-2 test against BF16 reference logits, with a fresh 48-chunk BF16 imatrix and Gram capture [E32].

The three arms are not byte-matched, and every statement below should be read against this table:

arm	IQ2 class	IQ3 class
GSQ-RCO, trained	IQ2_XS, 8.42 GB	IQ3_S, 11.77 GB
Unsloth baseline, untrained	UD-IQ2_S, 8.37 GB	UD-IQ3_S, 12.04 GB
baseline plus our correction	9.27 GB (8.37 + 0.90 adapter)	12.94 GB (12.04 + 0.90 adapter)
stock file near the corrected arm's bytes	UD-Q2_K_XL, 9.83 GB	none measured

Our arm is a rank-128 full-covariance correction with Q8_0 factors on the Unsloth baseline. It is larger than the GSQ-RCO file by 0.85 GB in the IQ2 class and by 1.17 GB in the IQ3 class. The re-rounder was not used, because its codec does not cover the I-quant formats.

The cost of our arm, end to end: the extraction takes about 12 minutes of wall time on the CPU per model (724, 727 and 744 seconds). That figure excludes obtaining the BF16 weights (54.7 GB) and the 48-chunk Gram capture, which needs our patched build and which the capture log estimated at about 22 minutes. The released checkpoints come from a GPU training run whose cost we did not measure [E32].

5.2 512-token KL divergence on wikitext

Paired per-chunk differences, wikitext-2 test, n = 40 [E32, E35]:

pair (A – B)	ΔKLD	t	A better on	Wilcoxon p
GSQ-RCO IQ2_XS vs UD-IQ2_S (8.42 vs 8.37 GB)	+0.0715 ± 0.0061	+11.68	0/40	2e-12
GSQ-RCO IQ3_S vs UD-IQ3_S (11.77 vs 12.04 GB)	+0.0152 ± 0.0018	+8.56	1/40	6e-11
corrected vs own base, IQ2_S (+0.90 GB)	−0.0122 ± 0.0019	−6.45	39/40	9e-12
corrected vs own base, Q2_K_XL (+0.90 GB)	−0.0055 ± 0.0008	−6.98	34/40	2e-8
corrected vs own base, IQ3_S (+0.90 GB)	−0.0021 ± 0.0004	−5.89	36/40	1e-6
corrected IQ2_S vs GSQ-RCO IQ2_XS (9.27 vs 8.42 GB)	−0.0837 ± 0.0073	−11.49	40/40	2e-12
corrected IQ3_S vs GSQ-RCO IQ3_S (12.94 vs 11.77 GB)	−0.0173 ± 0.0018	−9.48	38/40	9e-12
corrected IQ2_S vs UD-Q2_K_XL (9.27 vs 9.83 GB)	+0.0337 ± 0.0029	+11.59	0/40	2e-12

Three readings, in order of byte fairness. First, at matched bytes the untrained vendor baseline already has lower KLD than the trained checkpoint at both classes (rows 1-2). Second, our correction lowers its own base at all three rungs, at a cost of 0.90 GB (rows 3-5); top-1 agreement rises at the two 2-bit rungs (+0.7 points, t about +3.8) and not at IQ3_S (t = +0.5). Third, the gap between the corrected arm and GSQ-RCO (rows 6-7) is mostly the first effect: at the IQ2 class, 0.0715 of the 0.0837 (85%) is the baseline's lead and 0.0122 (15%) is the correction; at the IQ3 class, 0.0152 of 0.0173 (88%) and 0.0021 (12%).

Against the ladder the result is that of §4.3. The stock UD-Q2_K_XL file at 9.83 GB is better than the corrected IQ2_S arm at 9.27 GB by 0.0337 KLD (row 8). The two are 0.56 GB apart, so this is a near-equal-byte comparison and not an equal-byte one.

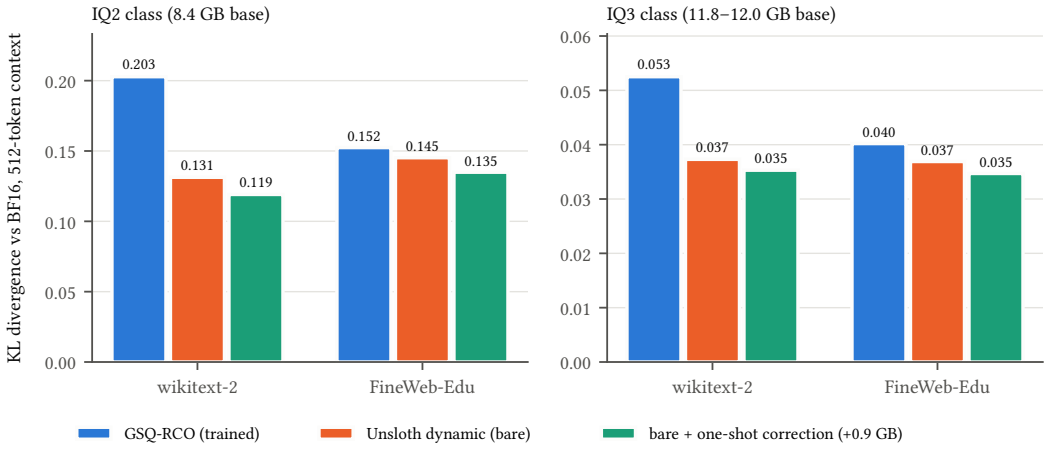


Figure 2: Released GSQ-RCO checkpoints of Qwen3.8-27B against their untrained Unsloth baselines and against the baseline plus a one-shot whitened correction, on two corpora (40 chunks each). The corrected arm carries a 0.9 GB adapter and is not byte-matched to the other two.

5.3 A second corpus, and both halves of a registered prediction

The wikitext result admits a calibration-mismatch reading: the released files may be tuned to text unlike wikitext. The calibration corpus of the released files is not documented. An earlier entry in our experiment log said they were trained on FineWeb-Edu; a check of the card and the released code the next day found no support for that, and the control was registered with the corrected wording: FineWeb-Edu is used because it is the one non-wikitext corpus the card reports on [journal].

The registration made a two-part prediction for the mismatch reading: the GSQ-RCO deficit against its baseline would shrink materially or change sign on FineWeb-Edu, and our own increments, whitened by wikitext Grams, would shrink too. The alternative reading predicted a GSQ-RCO deficit of $t \gtrsim +4$. The same eight contrasts on 40 chunks of FineWeb-Edu, against a BF16 reference on that text [E32, E35]:

pair (A − B), FineWeb-Edu, n = 40	ΔKLD	t	A better on	Wilcoxon p
GSQ-RCO IQ2_XS vs UD-IQ2_S	+0.0073 ± 0.0044	+1.66	13/40	0.022
GSQ-RCO IQ3_S vs UD-IQ3_S	+0.0033 ± 0.0016	+2.07	13/40	0.029
corrected vs own base, IQ2_S	−0.0103 ± 0.0011	−9.40	38/40	1e-11
corrected vs own base, Q2_K_XL	−0.0052 ± 0.0006	−9.20	38/40	5e-8
corrected vs own base, IQ3_S	−0.0022 ± 0.0003	−8.65	36/40	1e-7
corrected IQ2_S vs GSQ-RCO IQ2_XS	−0.0175 ± 0.0045	−3.94	35/40	1e-4
corrected IQ3_S vs GSQ-RCO IQ3_S	−0.0055 ± 0.0016	−3.39	31/40	3e-4
corrected IQ2_S vs UD-Q2_K_XL	+0.0471 ± 0.0044	+10.72	1/40	4e-12

The first half of the prediction held. The GSQ-RCO deficit falls about tenfold at the IQ2 class (+0.0715 to +0.0073) and fivefold at the IQ3 class (+0.0152 to +0.0033), to a difference that is unresolved by t at IQ2 and marginal at IQ3. On FineWeb-Edu the matched-byte comparison of trained checkpoint and untrained baseline is close to a tie.

The second half mostly did not hold. Our increments were predicted to shrink; the IQ2_S increment is 16% smaller (−0.0122 to −0.0103) and the other two are unchanged (−0.0055 to −0.0052; −0.0021 to −0.0022). The registered mismatch reading is therefore supported for the competitor's arm and not for ours, and we report it as a prediction that was half wrong. The useful consequence is a held-out measurement of the correction increment at 27B: a correction whitened on wikitext lowers its base on FineWeb-Edu by nearly the same amount.

With the baseline and the trained checkpoint close on this corpus, the corrected arm's lead over GSQ-RCO (−0.0175 and −0.0055) is about the size of the correction increment, which is bought with 0.9 GB. The stock UD-Q2_K_XL file again beats the corrected IQ2_S arm at near-equal bytes.

5.4 Context length against metric

The card's wikitext perplexities order GSQ-RCO ahead of the Unsloth baseline at both classes. At our 512-token context the paired NLL difference is unresolved: GSQ-RCO minus baseline +0.017 ± 0.013 (t = +1.33) at the IQ2 class and +0.003 ± 0.006 (t = +0.58) at the IQ3 class, n = 40. At a 2048-token context over the whole test set (145 chunks) the card's ordering reproduces: −0.041 ± 0.012 (t = −3.33) and −0.018 ± 0.004 (t = −4.15) [E32, E35].

That left two explanations for the disagreement between 512-token KLD and the card: context length, or the metric itself. A registered run separates them. We generated BF16 reference logits at a 2048-token context (40 windows; llama-perplexity scores the second half of each window, so every scored token has at least 1,024 tokens of context) and scored all eight files on them, recording KLD, NLL and top-1 agreement from the same run on the same tokens [E36]:

GSQ-RCO − Unsloth baseline, 2048-token context, n = 40	ΔKLD to BF16	Δtop-1 agreement	ΔNLL of the text
IQ2 class (8.42 vs 8.37 GB)	+0.0569 ± 0.0071 (t = +8.03, 0/40)	−2.37 pt (t = −8.88)	−0.025 ± 0.015 (t = −1.64)
IQ3 class (11.77 vs 12.04 GB)	+0.0178 ± 0.0035 (t = +5.02, 1/40)	−1.53 pt (t = −9.51)	−0.018 ± 0.007 (t = −2.49)

With long context the trained checkpoints are still further from the BF16 distribution than their untrained baselines, by about as much as at 512 tokens, and they agree less often with its top token; on the same tokens they assign the text higher likelihood (resolved at the IQ3 class, t = −2.49; leaning the same way at IQ2, t = −1.64). The registered rule reads this as a metric effect and not a context effect. Against the reference itself, the GSQ-RCO IQ3_S file's NLL on these windows is level with BF16′s (−0.004 ± 0.009, t = −0.46, ahead on 13 of 40), while every other file in the comparison, its baseline included, is resolved behind BF16 (t = +3.8 to +14.1) [E36]. A 3-bit file at KLD 0.061 that gives wikitext the reference's own likelihood has not recovered fidelity; it has moved away from the reference without losing likelihood on this text. Both published objectives are fidelity objectives (block-output reconstruction for GSQ, KL divergence to the full-precision model for RCO), and the calibration text of the released files is not documented, so we read the likelihood edge as an effect of the distribution the files were fitted on, and not as evidence of quality: for a post-trained reference, a perturbation can move perplexity in either direction ([43]). The evidence is one method (GSQ-RCO), one corpus at this context (wikitext), and one resolved class. On FineWeb-Edu at 512 tokens the trained files and their baselines are level in NLL (t = 0.00 and −0.91) as well as close in KLD (§5.3), and the card's own perplexities favour the baseline on C4 and FineWeb-Edu at the IQ2 class. An independent KLD table on the same files agrees with ours on wikitext and FineWeb-Edu and ranks the trained IQ3_S file closer to BF16 on a third, unpublished corpus (agentionai/Qwen3.8-27B-AP-GGUF model card), so the KLD ordering also turns with the corpus.

That KL divergence measures displacement from the reference and not its direction is not new. It is the thesis of [13], on GGUF quants scored on tasks; a quantization-aware-trained file with lower cross-entropy than a distilled one but far higher KL divergence is reported in [44]; and the GSQ-RCO authors separate task-lossless from distribution-lossless quantization ([45]). What this pair adds is a paired, matched-byte instance in which the two measures rank the same released files oppositely on the same tokens. The opposite case exists as well: a llama.cpp Q4_K file trained by distillation against the reference ends up closer to it than post-training quantization does ([46]), so a trained file being further from the reference is not a general rule.

The NLL record of our own arm is mixed, and we report it in full. At 2,048 tokens our correction moves every base closer to the reference (IQ2_S −0.0114, IQ3_S −0.0026 and Q2_K_XL −0.0081 in KLD; t = −6.7, −6.8, −5.3) and improves NLL at IQ2_S (t = −4.78) but not at IQ3_S (t = −0.22). In NLL at that context the GSQ-RCO IQ3_S file is ahead of our corrected IQ3_S arm (t = +2.43 for ours minus theirs), the reverse of the KLD ordering.

5.5 Capability

The card's claims are task-level. Before the final round we had one run per arm of the 112-unit fast subset of an agentic coding suite (multi-language tasks solved by an agent loop with a compiler and tests; single-slot serving, stock llama.cpp b10865, reasoning budget 20,480 tokens) on the two IQ3-class files: GSQ-RCO 88 of 112, the Unsloth baseline 76, a paired net of +12 (exact McNemar p = 0.012), against a four-run band of 82-91 for one unchanged file of a different finetune [E32]. A 9-unit spread for an unchanged file means a +12 difference between two single runs has to be read as one draw. The suite samples at temperature 0.6 without a fixed seed, so repeated runs

are independent draws. Agentic results on quantized files are known to move with prompts, scoring design and serving settings ([47]), and on llama.cpp prefix-cache reuse alone makes repeated episodes diverge more often at lower bit widths ([48]). Every run below uses the same harness, prompts, chat template and llama-server settings, prefix cache included, and the analysis pairs the arms unit by unit.

[PENDING:F — three runs per arm in the current eval era: GSQ-RCO IQ3_S, UD-IQ3_S, and UD-IQ3_S + our correction; per-unit pass rate paired over 112 units (Wilcoxon, sign test), run-level mean ± SD; registered rule in PLAN.md §F.]

5.6 What the head-to-head shows

[PENDING:F — final paragraph once the capability replication is in.] Against released GSQ-RCO checkpoints, the untrained vendor baseline at matched bytes is closer to the BF16 distribution on wikitext at both 512 and 2,048 tokens and close to tied on FineWeb-Edu; the trained checkpoints assign wikitext higher likelihood at 2,048 tokens, at the IQ3 class as high as the reference itself. Our correction (+0.9 GB, not byte-matched) moves its base closer to the reference on both corpora at 512 tokens and on wikitext at 2,048 tokens, with a resolved NLL gain at IQ2_S and none at IQ3_S, and a stock file at near-equal total bytes beats the corrected arm. For these files, KL divergence to a reference ranks fidelity to it and not likelihood of the text. It suits a method that only approximates the reference, such as ours; for ranking a trained file against an untrained one it needs a likelihood or task measure beside it.

6. Serving: the adapter's decode cost and how much of it kernels remove

6.1 The cost on stock llama.cpp

Every corrected configuration in §4 and §5 carries a decode cost that a bare quantized file does not. On stock llama.cpp the adapter path adds two unfused matrix-vector launches per projection. At batch 1 this costs 33-39% of decode speed at 0.6B (667 → 444 and 407 tok/s). At 27B it costs 33% with F16 rank-64 factors (99.7 → 66.5 tok/s) and 19% with Q8_0 factors (99.7 → 81.0) [E4, rollup, E35]. Others report the same pattern: adapter operations at batch 1 are bound by launches and not by rank ([49]; [50]; [51]; CALDERA's Table 6, [5]). A corrected configuration that is competitive with a rung on bytes is therefore not competitive with it on stock llama.cpp today.

The kernels below remove most of that cost on local, unmerged patches. They leave perplexity digit-identical and each has a switch to disable it. All speed comparisons are same-session A/B pairs, because drift between sessions on identical configurations reached 9% and invalidated one number in our first tables (86.3 tok/s, 82.3 on re-measurement) [review].

Evidence grade. Only one speed result in this section has raw benchmark output in the supplementary material: the N = 10 interleaved pair at 27B (§6.2). Every other speed number is transcribed from the per-phase reports, which record the measurements as prose tables. All of them come from one GPU and one host, at one pinned llama.cpp build that predates b10865. The patch set has since been rebased onto b10865 and no speed number was re-measured after the rebase.

6.2 Three phases of kernel fusion

Window fusion. The five-node subgraph llama.cpp emits per adapted projection (W·x, A·x, B·t, scale, add) is matched in the CUDA backend and collapsed to two kernels. The result was +10% at rank 16 and no change at rank 64: removing 588 of 784 extra kernel launches per token did not move rank-64 decode, so launch count was not the bottleneck on an idle GPU [E14].

Folding A into the activation-quantize kernel. Following the design of SVDQuant ([52]), we fold t = A·x into the existing activation-quantize launch and B·t into the matrix-vector epilogue. Rank-64 decode at 0.6B went from 476 to 685 tok/s (+44%), which reduced the adapter cost in that session from 45% to 20%. The constant cost had been serialization: the epilogue loaded B and t after the main dot-product loop, adding a memory-latency round to every kernel. Loading static operands before the grid-dependency synchronization point removes it [E14].

Quantized factors and register pressure. Accepting Q8_0 factors in the fused paths gave +62% at 0.6B (394 → 638) and at first slowed 27B by 6.6%, because unfused Q8_0 factors already use an int8 kernel upstream. Two size gates restored +0.4% at 27B [E14]. The remaining 27B cost was register pressure: the fused kernel carried about 19

extra registers (61 ⟶ 80), which lowered occupancy. Moving the adapter dot product ahead of the main loop, with its result held in shared memory, restored 62 registers. Same-session pairs after that change: 0.6B 391 ⟶ 702 tok/s (+79%, one pair); 27B 84.7 ⟶ 86.9 (+2.6%, one pair) [E14].

Re-measured on a quiet host with interleaved blocks (N = 10, median and interquartile range), the 27B corrected configuration decodes at 88.1 [88.0-88.3] tok/s fused against 85.8 [85.7-85.8] unfused: +2.7%, against a measured ceiling of +4.9% for any epilogue design [E27, E14].

Two practices came out of this that may transfer to other memory-bound matrix-vector kernels: load static operands before the grid-dependency synchronization, and keep the epilogue from adding registers to the main loop by holding its state in shared memory.

6.3 An allocator fix for stream concurrency

llama.cpp's CUDA graph optimization was rejecting stream concurrency on adapter graphs, correctly: the allocator keeps one global free list, so a buffer freed in one branch of a fork can be handed to a tensor in a concurrent branch. Our change pins buffers within a fork-join region: frees inside a region are deferred to the join, and an in-place guard keeps the last consumer of a forked tensor from taking a buffer another stream still reads. The backend test suite (12,996 cases) passes, perplexity is digit-identical, and 200-token greedy generation is string-identical with the change on and off on three configurations.

The speed result is modest and was measured on a loaded host (load 40-66 on 32 threads throughout). At 0.6B, base-model decode rose in two sessions by 11% (763 ⟶ 849 tok/s) and by 19.6% (794 ⟶ 950); the report calls the second the best pair. With an adapter the gain was 7-12%. At 27B, six alternated sessions showed no difference outside noise: streams cover only the one-in-four full-attention layers of the hybrid architecture, and the large matrix-vector products already fill the GPU [E25]. The change does not depend on adapters.

6.4 Where the decode cost stands

At 0.6B the adapter cost on the patched build is about 20%, down from 45%. At 27B the fused corrected configuration (88.1 tok/s) is within the +4.9% ceiling of the unfused one, so with Q8_0 factors little of the remaining cost is in the epilogue. What remains against a bare rung is the adapter's VRAM and the per-token chain of extra nodes, which published work prices at 1.3-2 μs per node ([53]). None of the patches is merged upstream.

7. Limitations and open work

7.1 Limitations

Models and hardware. Five of the six models are Qwen-family: Qwen3-0.6B, Qwen3.5-0.8B, Qwen3.6-27B (as the heretic-v2 finetune), Qwen3.8-27B and Qwen3.6-35B-A3B. Llama-3.1-8B is the one model outside the family; on it the re-rounding result and the I-quant comparison replicate, and the correction does better against the adjacent K-quant than on Qwen (§3.1, §4.3). One non-Qwen model does not establish generality across families. Every measurement comes from one GPU (an RTX 5090) on one workstation.

Baselines not compared. The re-rounder is compared with the file it starts from, with stock rungs above and below it, and with a free-grid GPTQ baseline at identical format and bytes (§3.4). It is not compared with learned or gradient-tuned rounding (AutoRound exports such files for the K-quant formats; [28]), or with quantizers that add incoherence processing, trained codebooks or trellis codes (QuIP#, [54]; AQLM, [55]; QTIP, [56]). llama.cpp's own I-quants are lattice-codebook quantizers (E8 for IQ2_XXS and IQ2_XS, D4 for IQ3_XXS) without incoherence processing, and they are in every comparison here; trellis types exist as GGUF in the ik_llama.cpp fork and were not run. The type-allocation controls were built by a fixed promotion rule and were not compared with systematic allocation methods (EvoPress, [57]; a probe-based K-quant allocator on Qwen3.8-27B, [35]). The correction is the symmetric kind; an asymmetric, output-Fisher-weighted compensator ([12]) was not run against the controls.

Distribution. Calibration is on wikitext-2 train unless stated. The comparisons of §3.1 (at 0.8B, 8B and 27B) and §4.3 are also scored on FineWeb-Edu and a code corpus that no calibration touched; §4.5 and §5.3 have FineWeb-Edu only; the 2,048-token comparison of §5.4, the 0.6B rows and the mixture-of-experts table have no held-out corpus. The held-out scoring changed two conclusions: the 27B margin over Q3_K_S (§4.3) and the generality of

re-rounding (§3.1). Two corpora are not a benchmark of distribution shift; both held-out corpora are English text or code from public sources, and the code corpus is small (400 KB).

Statistics. The t statistics quantify evaluation-chunk variance on one corpus at a time. Calibration-sampling variance was measured once, at 0.8B, and is small against the effects reported (§3.1); it was not measured at 8B or 27B, where a different calibration draw has been enough to reverse a method ordering in other work ([58]). No family-wise correction is applied; the paired tables carry enough contrasts that margins near |t| = 2-3 are suggestive. The 100-chunk wikitext 27B comparison is a reproduction on the same chunks, not an independent sample; chunks 101-580 of the whole-test-set run (§4.3) and the held-out corpora are the independent samples. Blocks in the block bootstrap are runs of consecutive chunks and not articles.

The instrument. KL divergence to the reference measures fidelity to it. That is the right measure for a method that only approximates the reference, and it does not rank a trained artifact against an untrained one (§5.4). How KL divergence converts into answer flips varies across models and corpora, and total variation tracks flips more closely ([59]); every KLD comparison here is paired within one model and corpus, and the off-distribution ratios of §4.3 and §4.5 compare across corpora and inherit that caveat. Task-level evidence remains limited: one multiple-choice benchmark at one byte count (§4.3), and one coding suite on three IQ3-class files (§5.5). None of our re-rounded files was run on the coding suite.

Controls. Mixed-type controls now exist at 0.8B (one by the registered rule, one byte-matched after registration), at 27B and on the mixture-of-experts model; each is a single designed point, built by a rule and not optimized. None was built at 0.6B.

Width. The regression of §4.2 has one step on the width axis and its intervals resample tensors, not models. Llama-3.1-8B (d = 4,096) was not added to it, because it would change family and width at once.

References and precision. Rows labeled legacy are scored against a Q6_K or Q8_0 file and not BF16 (measured contamination at 27B: 0.0032 KLD). The mixture-of-experts table is scored against a vendor Q4-class file at 20 chunks. At 27B, "full covariance" means block-diagonal for ffn_down (§2.2), the VRAM columns were not measured, and the rank allocation used sensitivity priors from 0.6B. Rows from before the Gated-DeltaNet normalization fix in llama.cpp (b10829) are paired within one build and never tabled beside current-build rows; every hybrid-architecture comparison in §3.1, §4.3, §4.5 and §5.4 was re-scored on the current build.

Alternation. The ordering claim of §3.3 is paired (t = +17.6). The claim about alternation rounds is weaker: the 0.6B sequence is unpaired, the 0.8B continuation has no recorded error bars, and the 27B gain from alternation is about one unpaired standard error.

Speed. One GPU, one host. Session-to-session drift reached 9%; only one speed result has raw output in the supplementary material; nothing was re-measured after the patch set was rebased (§6.1).

Verification rests on our own records. Statements that a derivation was verified numerically, that a capture passed its gates, or that an experiment was registered before it ran cite our review notes and experiment log. Those are our records and have not been checked by anyone else. The registration for the experiments added in the final round was committed to version control, with timestamps, before the runs; the history will be linked in the de-anonymized version.

Code hazards. The per-tensor caches of the re-rounder, the extractors and the spectra tools are keyed by tensor name only and carry no provenance fingerprint; in the final round every run used a fresh cache directory per arm, and the Llama-3.1-8B pipeline also stamped each cache with its inputs; the tools themselves are unchanged. Column selection under Gram whitening ranks the wrong columns (the reported column runs predate full-covariance whitening) [review].

How the work was done. Experiments, analysis and drafting were carried out by AI research agents under the author's direction, on one workstation. The author is responsible for the claims; errors found in the record and their corrections are listed in §7.6.

7.2 Status of the ablation program

An audit of every cell behind this paper's claims counted 51: 17 clean, 15 confounded, 19 missing, and priced seventeen runs to close the gating cells [ablation]. All seventeen have now been run except the 0.6B half of the small-scale controls, and a referee's further requests have been run as well [E36]:

item	where
calibration-sensitivity grid (five calibrations × two methods × three corpora)	§3.1, §4.8
calibration-sampling bound	§3.1
mixed-type controls at 0.8B	§4.3
held-out pass over the 27B ladder comparisons	§4.3
recovery curve with the recipe fixed	§4.5
equalization without an imatrix, and a strength sweep	Appendix A.3
27B re-rounder under single-step provenance, with IQ2_M and IQ3_XXS brackets	§3.1
KL divergence at a 2,048-token context for the head-to-head files	§5.4
free-grid GPTQ baseline for the re-rounder	§3.4
one model outside the Qwen family (Llama-3.1-8B)	§3.1, §4.3
three capability runs per IQ3-class arm, and our corrected arm	§5.5 [PENDING:F]

Two diagnostics were added after the 27B re-round failed its registered test, each registered before it ran: the share of the file's KLD carried by the tensors the sweep covers, and the same sweep with the earlier Gram capture (§3.1).

Not run: a mixed-type control at 0.6B (the 0.6B rows are legacy); the mixture-of-experts table against a BF16 reference (none is available to us); learned-rounding and vector-quantizer baselines; a third width inside one family.

7.3 Reproducibility and artifacts

Method descriptions, scripts, raw logs, per-chunk outputs and result tables are in the supplementary material, one directory per experiment; Appendix B maps each tag used in this paper to its directory. The statistics added in the final round are produced by one script from existing logs [E35]. Large binaries (reference logits, Gram captures, quantized models and adapters, about 160 GB) are not included. They can be rebuilt from the recorded recipes; some early ones were deleted to reclaim disk, so a number resting on a deleted artifact is reproducible only by rebuilding it. Two reproduction gates were run: one rebuilt the 0.8B base with 335/335 tensors byte-identical and re-derived the re-rounded file byte-identical, and one reproduced the published 0.8B numbers before any new comparison [E26].

Three components are not stock llama.cpp: the Gram-capture extension to llama-imatrix (about 140 lines, environment-gated), the llama-gradmatrix tool, and the fused-kernel and allocator patch set (numbers reported at a build that predates b10865; since rebased onto b10865). The re-rounder and the extractors are standalone Python against gguf-py. Serving needs none of the patches: a re-rounded file is a standard GGUF, and a corrected model is a standard GGUF plus a standard LoRA-form adapter. Producing either needs the Gram-capture patch and the source weights.

7.4 Open work

Re-rounding at size. Before more codecs (Q3_K, Q4_K, the I-quants, NVFP4; [60]), the 8B and 27B failure needs an explanation: §3.1 shows the sweep optimizing its proxy while the outcome does not move. Candidate tests are the published repairs for the same failure applied on K-quant grids, a penalty on drift from the original weights ([61]) and a worst-case objective over activation distributions ([8]); a metric estimated from far more calibration text, or shrunk toward its diagonal; a rotation before re-rounding; and an end-to-end objective in place of the layer-local one. Any packaged tool would need a held-out check before release, since a re-rounded file can be worse than the original on text its calibration missed [wins].

Estimators for two-sided metrics. Following §3.2: damping the output-side factor toward the identity, multi-pass gradient captures, and an adjoint for the GDN layers to remove the gradient cut. An output-Fisher target estimated over many samples has made a two-sided correction work on a 2-bit base ([12]); the same target against the same-byte controls of §4.3 is the most direct test of whether a stronger correction changes the verdict.

Other directions. Riemannian refinement of the closed-form factors and the Bregman-damped metric-modification scheme of [62] [manifolds] per-workload conditioned adapters (§4.8); a loader that aliases shared factors (§4.4); correction of the dense tensors of a mixture-of-experts model.

7.5 Summary

Re-rounding a Q2_K file's codes under an activation-covariance metric lowers its KL divergence by a third at 0.8B and, as a one-shot sweep on an unrotated grid, gives no resolved gain at 8B and 27B, where it halves its own objective on tensors that carry most of the file's divergence and still damages held-out text; at every size where we scored held-out text, it damages kinds of text its calibration does not cover. Refitting the grid beats freezing it, and does not change that verdict at size. Low-rank correction improves its base on every corpus we scored, by less off the calibration distribution in the ladder and recovery comparisons (0.25 to 0.96 of the wikitext gain; 0.84 to 1.05 on the head-to-head files) and by more the thinner the base. Against the stock types it falls short: an I-quant type at fewer bytes beats it at 8B and 27B on every corpus, its lead over the adjacent K-quant at 27B exists only on the calibration distribution (on Llama-3.1-8B it holds on both prose corpora), and a same-byte mixed-type control beats it on every corpus at 27B and by a factor of three at 0.8B. Three placements of the low-rank bytes could not be distinguished within a stated bound. Against one family of released trained checkpoints, KL divergence to the reference and likelihood of wikitext disagree in the same run, so for those files the comparison measures what each file optimizes rather than which is better; [PENDING:F — capability replication]. The kernel and allocator patches reduce the adapter's decode cost at small scale and change little at 27B. The most transferable results are methodological: held-out corpora reversed the 27B ladder margin and exposed the cross-domain damage of re-rounding; single-step provenance erased the earlier 27B re-rounding gain; same-byte controls outranked every corrected configuration on Qwen; and a surrogate that improved failed to predict the outcome seven times (§4.7).

7.6 Corrections made to the record

The project record contains errors that were found and corrected before this paper, and they are listed here so that a reader of the supplementary material can find them.

- A first set of 27B comparisons, unpaired and against a requantized reference, showed a margin for the corrected configuration that reversed sign under single-step provenance (§4.3).
- A set of paired t statistics for the head-to-head of §5 was inflated by dividing per-chunk differences by a per-token standard error. They were recomputed with the canonical tool before any conclusion changed; the corrected values are the ones in §5.2.
- Two capability rows were retracted when 84-87% of their failures turned out to come from a crashed server.
- A first "mixed-calibration" run never reached its second corpus, because the two corpora were concatenated and not interleaved.
- An early statement that capture falls as r/d rested on two points; §4.2 replaces it with a fit in which r/d is the worst of six forms.
- A journal entry asserted that the GSQ-RCO checkpoints were trained on FineWeb-Edu; the card and code do not say so (§5.3).
- Every hybrid-architecture row in an earlier draft was scored on a llama.cpp build that predates its Gated-DeltaNet normalization fix. The paired comparisons were valid within that build; this paper re-scores them on the current build and reports both where they differ (§4.3).
- An earlier draft stated that at 0.6B and 0.8B "controls beat crowns" by inference from interpolation between rungs; §4.3 now measures it at 0.8B.
- An earlier draft reported re-rounding at 27B as a 13.3% KL-divergence gain. That pair was legacy provenance; under single-step provenance the gain is 3.7% and unresolved, and held-out text is harmed (§3.1).

Each is kept in the journal with its correction [journal, review].

8. Related work

Low-rank correction of quantized weights. Adding a low-rank branch to a quantized base begins with LoRC in ZeroQuant-V2 ([1]), an unweighted rank-8 SVD. LQER ([2]) observes that the raw residual has a Marchenko-

Pastur spectrum, so the value lies in a weighted decomposition. QERA ([3]) supplies the mathematics we use: its Theorem 1 is the full-covariance whitened closed form, and the diagonal special case in its Theorem 2 is the statistic llama.cpp's imatrix computes. EoRA ([4]) is the nearest industrial counterpart, and CALDERA ([5]) contributes the alternation with best-iterate tracking and reports, in its own tables, the fixed-rank benefit shrinking with model size. What this paper adds to that line is the setting: the same pipeline measured with diagonal and full whitening on llama.cpp's deployed K-quant and I-quant formats at 0.6B to 35B, compared against the format's own ladder and against mixed-type controls at matched bytes at 0.8B, at 27B and on a mixture-of-experts model, which the correction loses to (§4.3, §4.6), on and off the calibration distribution. The newest member of the lineage, a two-stage compensator with an asymmetric, output-Fisher-weighted target ([12]), argues that the symmetric target the lineage solves is itself high-rank and recovers much more of the held-out gap on a 2-bit base; it builds no same-byte control, and our verdicts are scoped to the symmetric kind (§4.3).

Rank and width. The decay we fit in §4.2 has published counterparts: CALDERA's Theorem 4.1; the lower bound of [20]; LRC ([63]), where rank at 10% of width halves the W4A4 gap; ResQ ([64]), which fixes rank at an eighth of width; FLRQ ([65]), whose allocated ranks beat a uniform rank at fewer bytes; and TwinQuant ([66]), which reports flat spectra beyond rank 256. RILQ ([6]) reports that 2-bit error is high-rank and that layer-local correction saturates.

Re-selecting codes on a fixed grid. GPTQ ([7]) is the source of our sweep. ReQuant ([10]) performs cyclic coordinate descent on the same quadratic for generic post-training quantization, not GGUF. DRQ ([8]) refines codes on a frozen grid, reports that lower calibration reconstruction loss raises held-out perplexity, and repairs it with a worst-case objective over activation distributions; SARQC ([61]) penalizes drift from the original weights for the same reason; JARQ ([31]) shows that refitting the grid to the codes beats refining codes or scales alone. Inside the GGUF formats, AutoRound exports gradient-tuned K-quant files ([28]), and IST-DASLab's gptq-gguf-toolkit (2025) runs GPTQ with the K-quant grid refit at each superblock, the free-grid arm of our §3.4. SchurQuant ([11]) also refits the scales, and reports that layer-wise reconstruction is a loose surrogate for the final loss, which agrees with §4.7. GSQ ([9]) is the other method released as GGUF files: Gumbel-Softmax relaxation trained against block-staged reconstruction and projected back into the format. Our re-rounder is a one-shot, CPU-only member of this group that works on the K-quant block hierarchy; §3.4 compares it with GPTQ allowed to refit the grid inside the same format. §5 compares released GSQ-RCO checkpoints with our correction, not with our re-rounder, whose codec does not cover I-quants. Our placement of these methods relative to one objective is argued in a project note [theory-L6].

Three neighbouring classes of work are not compared in this paper: learned or adaptive rounding on a fixed grid, which is the older form of "re-choose only the codes"; quantizers that add incoherence processing, trained codebooks or trellis codes to vector quantization (QuIP#, AQLM, QTIP), where llama.cpp's I-quants are the lattice-codebook member without incoherence processing; and systematic mixed-precision allocation, the version of the type-allocation control we built by hand. §7.1 lists these as limitations.

Richer metrics. GuidedQuant ([67]) reaches strong 2-bit results with a block-diagonal end-loss Fisher and no correction branch. OBD-LLM ([17]) applies a two-sided Kronecker-whitened objective to residual correction after GPTQ; ours is its one-sided case. We stay one-sided for the reasons given in §2.2. KronQ ([68]) and ARHQ ([69]) are further variants of the metric that we did not test.

Placement and allocation. §4.4 compares post-hoc correction with a pre-quantization carve in the order of SRR ([37]) and with shared bases in the arrangement of GlowQ ([38]), and could not distinguish them within its bound. AWSRC ([40]) uses seeded bases; in our whitened geometry such bases capture what a random subspace would (§4.4). Per-tensor type allocation is what RCO ([39]) optimizes with an exact-budget dynamic program, and EvoPress ([57]) by evolutionary search; on Qwen3.8-27B a probe-based allocation of llama.cpp K-quant types beats the stock IQ3_M mix at near-equal bytes, and the same work finds round-to-nearest error spectrally flat ([35]). At a matched budget, spending bits globally beats repairing the most recoverable layers locally ([70]), which is the allocation-side counterpart of our control result. FLRQ and the integer program of LQ-LoRA ([71]) allocate rank and bits in the same spirit.

Mixture-of-experts. MiLo ([72]) uses our architecture on such models, a quantized base with low-rank compensators, and states that its compensator is unfused. TileQ ([73]) tiles within experts, consistent with our finding that

pooled expert residuals have no shared subspace (§4.6). The router-guided compensators of [74] spend computation that a byte-bound setting does not have.

Serving. Fused decode paths for correction branches now exist for GPU 4-bit kernels: SPEAR fuses a gated low-rank compensator into the GEMM epilogue at batch 1 (+25% over the bare kernel at 1B, +6-14% at 70B; [75]), and TwinQuant and FLRQ report 4-8% overhead; DecDEC fuses a non-low-rank residual at 1.7% on a laptop GPU ([76]); SVDQuant ([52]), whose fusion design our kernels follow, ships for diffusion models. EoRA ships unfused in HF/vLLM, CALDERA's Table 6 records the decode loss, and MiLo's compensator is unfused. None targets llama.cpp's K-quant and I-quant formats, where the stock adapter path has no fused decode (an open upstream issue, #27093, reports about 27% token-generation loss from a rank-1 adapter). §6 measures the cost on llama.cpp because a correction that decodes 20-40% slower than its bare base is a different artifact from that base.

The upstream discussion. llama.cpp discussion #8831 proposed an LQER-style correction in 2024 and was met with the prediction (ik_llama.cpp #15) that mature quantizers would not benefit. That thread also posed the same-byte question in so many words, showed a 4.30-bpw L²QER result trailing IQ4_XS at 4.25 bpw, and reported an informal test on a 270M model in which a carve plus correction was no better than the stock Q2_K mix. Our measurements are the controlled version of that test, and agree with it for correction against the ladder. Two observations made during this work, about the imatrix on multi-token-prediction layers and about backpropagation through the KV cache, had already been reported upstream (#23476, #23575; #21037) and are not claimed as new.

Measurement. We score files by mean KL divergence against reference logits with paired per-chunk statistics, following the observation that KL divergence tracks answer flips ([22]) and llama.cpp practice (#4110), with perplexity secondary. Total variation tracks flips more closely than KL divergence across models and corpora ([59]). That KL divergence measures displacement from the reference and not its direction is argued for GGUF quants in [13], and for trained against distilled files in [44]; §5.4 gives a paired, matched-byte instance in which KL divergence and likelihood rank the same released files oppositely, and we report it as a limit on the measure.

Broader Impact Statement

This work studies how to store large language models in less memory at a given quality, so that larger models run on consumer hardware. The direct effect is to lower the cost of running models locally, which widens access and keeps data on the user's machine; it equally lowers the cost for any user, including one who intends misuse. The methods change only how existing weights are stored and do not add capability beyond the source model. One of the models we measure (heretic-v2) is a publicly released community finetune with safety refusals removed; we use it only as a quantization target, report distribution-level fidelity metrics and a coding benchmark, and release no new model weights. We see no risk specific to this work beyond those of model compression in general.

Appendix A. Additional negative results

These results are summarized in the main text and given here in full.

A.1 A correction on an NVFP4 base at 0.8B

NVFP4 is a 4-bit floating-point block format. A whitened low-rank correction on a frozen NVFP4 base of Qwen3.5-0.8B improves that base on 40/40 chunks (KLD 0.207 → 0.104, top-1 +6.2 points) and loses to the ladder: the 603 MB composite is behind every stock rung at or below its bytes that we scored (Q4_K_S at 519 MB, 0.047; Q5_K_S at 578 MB, 0.015). Under llama.cpp's quantizer, which has no importance weighting specific to NVFP4, the bare NVFP4 file is the worst 4-bit option on this architecture (0.207 against 0.088 for Q4_0 at equal bytes). Its residual is slightly more capturable than that of Q3_K_M (capture 0.736 against 0.721), yet the correction recovers less of the damage (50% against 70%). This is row 6 of the table in §4.7 [E34].

A.2 The thinnest base: IQ1_S with large factors

This test takes the idea of replacing base bits with factors as far as llama.cpp's formats allow. The base is IQ1_S, the thinnest type available (nominally 1.56 bits per weight; the 7.44 GB file is 2.18 bits per weight by file bytes). With rank 512 on non-FFN tensors and rank 256 on FFN tensors (2.52 GB of Q8_0 factors), recovery of the base's

KLD damage rises to 52%, and the result still loses to bare Q2_K at near-equal total bytes: perplexity 10.50 and KLD 0.431 at 9.96 GB against 7.891 and 0.153 at 10.86 GB. These are legacy 27B rows, scored against a Q6_K reference.

The exchange rate is the point. Going from 0.80 GB to 2.52 GB of factors lowered KLD from 0.527 to 0.431, which is 0.056 per GB. Base bits bought 0.216 per GB (IQ1_S to Q2_K) or 0.323 per GB (IQ1_S to IQ2_XS), four to six times as much [E15, E35]. Published work on real factorization ratios reports retraining budgets above 50 billion tokens ([77], [78]); a calibration-only correction does not substitute for that.

The corrected IQ1_S file with the smaller adapter (8.24 GB on disk, KLD 0.891 → 0.527, top-1 +8.5 points, 16.5 unpaired standard errors) sits where we measured no rival rung. The equal-byte rivals IQ1_M and IQ2_XXS were not built, and 8.24 GB is file size; VRAM would be about 10.3-10.5 GB on the measured adapter-overhead pattern [review].

A.3 Equalization before quantization

The AWQ and SmoothQuant line of work moves importance into the weights by scaling columns so the quantizer spends its range where activations are large. We tested the full-strength version on llama.cpp's imatrix-weighted quantizer. On this architecture only diagonal scalings fold exactly, because every candidate site sits behind a normalization with a learnable gain; restricting the scales to powers of two makes the fold bit-exact (equalized BF16 against stored BF16: KLD 0.000000, top-1 100%).

Requantizing the equalized model with the same recipe at the same bytes is strongly harmful: perplexity 33.09 → 41.73, paired ΔKLD t = +28.3 (n = 40, equalized better on 0 chunks), ΔNLL t = +59.8 (n = 580). The quantizer's own weighted error rises 6-68% per folded tensor and is unchanged on the untouched control tensors [E26].

Two ablations registered for the final round separate the candidate mechanisms, on the current build against BF16 references, 40 chunks per corpus [E36]. Strength: scaling the fold by α (each power-of-two column scale's exponent multiplied by α and rounded, so the fold stays exact) gives ΔKLD against the file as written of +0.022, +0.068, +0.075 and +0.198 on wikitext at α = 0.25, 0.5, 0.75 and 1 (t = +5.1 to +29.2). On FineWeb-Edu the ordering is the same (+0.010 to +0.175); on code α = 0.5 and 0.75 are level (+0.123, +0.121) and the rest are ordered. There is no interior optimum: every strength tested harms, and the harm grows with strength. Imatrix: equalization at α = 1 without an imatrix, against the file quantized without one, still harms (+0.305 on wikitext, t = +16.5; +0.186 and +0.607 on the held-out corpora). If the harm came only from counting importance twice, once in the scales and once in the quantizer's weights, it would vanish without the imatrix; it does not. Relative to its own bare file it is smaller without the imatrix, though (+18% of the bare file's KLD against +40% with it), so a double-counting component is not excluded.

The mechanism we can support is a format interaction: the fold puts up to 32× dynamic range into 16-element sub-blocks whose shared 4-bit sub-scales cannot span it, and smaller strengths put in less of it. The result is scoped to one format and one architecture. This is row 4 of the table in §4.7.

A.4 Further descent of the same objective

A natural extension of single-pass re-rounding alternates the code sweep with a per-row least-squares refit of the format's storable scales, both steps descending the same whitened objective, with the 4-bit sub-scales frozen so the file stays byte-compatible. Run to a pre-registered cap of 8 iterations at 0.8B, the alternation lowers the whitened objective 2.6% below single-pass re-rounding. End to end nothing changes: paired ΔNLL +0.0009 ± 0.0022 (t = +0.40), ΔKLD +0.0015 ± 0.0034 (t = +0.43), Δtop-1 −0.55 ± 0.42 points (t = −1.32). The same harness resolves re-rounding against the untouched file at t = −30.3 (KLD) and −44.9 (NLL) [E26].

No tensor reached a fixed point of the codes. Of 96 tensors, 92 ran to the cap and 4 stopped earlier on a plateau rule; the objective falls for three or four iterations and then enters a cycle of about 0.1% amplitude. Single-pass re-rounding is therefore the recipe we recommend, and this is row 3 of the table in §4.7.

A.5 Alternation: the termination argument and its amendment

The alternation of §2.4 keeps the best iterate and stops when a round is worse. Our theory note makes three claims about it [theory-alt]. First, the quantizer's reachable output set is finite and the corrector step is an exact minimum, so the best-iterate value is non-increasing over a finite set and becomes constant after finitely many rounds. Second,

the first round improves exactly when subtracting the corrector steers the quantizer to a grid point whose residual is more compressible at rank r in the whitened metric; this was observed at both small sizes (whitened capture $0.37 \longrightarrow 0.46 \longrightarrow 0.51$ across rounds at 0.6B). Third, and informally, per-round gains decay as the composed map approaches a fixed point.

The third claim needed an amendment. When a related alternation was pushed to its 8-iteration cap at 0.8B (A.4), no fixed point of the codes was reached; the iterates settle onto a small limit cycle. The best-iterate argument is unaffected. The quantizer's internal objective differs from the whitened one, which bounds what alternation can reach; schemes that modify the quantizer's metric address this ([30]; [62]). In practice we used two or three rounds, and changes after the second were below noise.

Appendix B. Artifact index

Bracketed tags in the text name the place in the project repository where the measurement, its raw logs and its scripts live.

tag	path
E1	`research/lowrank/experiments/01-svd-spectrum/`
E3	`research/lowrank/experiments/03-residual-rank/`
E4	`research/lowrank/experiments/04-served-v0/`
E10	`research/lowrank/experiments/10-full-covariance/`
E13	`research/lowrank/experiments/13-rerounder/`
E14	`research/lowrank/experiments/14-fused-kernel/`
E15	`research/lowrank/experiments/15-iq1-carrier/`
E17	`research/lowrank/experiments/17-task-conditioned/`
E20	`research/lowrank/experiments/20-gradgram/`
E23	`research/lowrank/experiments/23-moe/`
E24	`research/lowrank/experiments/24-yaqa-lite/`
E25	`research/lowrank/experiments/25-alloc-concurrency/`
E26	`research/lowrank/experiments/26-lloyd-gauge/`
E27	`research/lowrank/experiments/27-bf16-rederivation/`
E28	`research/lowrank/experiments/28-glowq-shared-a/`
E29	`research/lowrank/experiments/29-srr-split/`
E31	`research/lowrank/experiments/31-seeded-basis-duel/`
E32	`research/lowrank/experiments/32-t117-gsq-head-to-head/`
E33	`research/lowrank/experiments/33-t110-capture-width/`
E34	`research/lowrank/experiments/34-e16-nvfp4-08b/`
E35	`research/lowrank/experiments/35-final-reanalysis/`
E36	`research/lowrank/experiments/36-submission/`
ablation	`research/lowrank/paper/ABLATION-PLAN.md`
journal	`research/lowrank/JOURNAL.md`
manifolds	`research/lowrank/prior-art/MANIFOLD-CANDIDATES.md`
protocol	`research/lowrank/PROTOCOL.md`
review	`research/lowrank/review/`
rollup	`research/lowrank/RESULTS_ROLLUP.md`
theory-L6	`research/lowrank/paper/theory-L6-family-subsumption.md`
theory-alt	`research/lowrank/paper/theory-alternation-convergence.md`
wins	`research/lowrank/paper/DEPLOYABLE-WINS.md`

References

Authors, titles and dates are as recorded by arXiv (`refs.json`, regenerated by `fetch_refs.py` from the saved API responses); a venue is given where the arXiv record or the project bibliography (`research/lowrank/paper/references.md`) states one. The project bibliography also records what each source was used for.

[1] Zhewei Yao, Xiaoxia Wu, Cheng Li, et al. ZeroQuant-V2: Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation. arXiv preprint, 2023. arXiv:2303.08302.

[2] Cheng Zhang, Jianyi Cheng, George A. Constantinides, Yiren Zhao. LQER: Low-Rank Quantization Error Reconstruction for LLMs. ICML, 2024. arXiv:2402.02446.

[3] Cheng Zhang, Jeffrey T. H. Wong, Can Xiao, et al. QERA: an Analytical Framework for Quantization Error Reconstruction. ICLR, 2025. arXiv:2410.06040.

[4] Shih-Yang Liu, Maksim Khadkevich, Nai Chit Fung, et al. EoRA: Fine-tuning-free Compensation for Compressed LLM with Eigenspace Low-Rank Approximation. ICLR Workshop, 2026. arXiv:2410.21271.

[5] Rajarshi Saha, Naomi Sagan, Varun Srivastava, et al. Compressing Large Language Models using Low Rank and Low Precision Decomposition. NeurIPS, 2024. arXiv:2405.18886.

[6] Geonho Lee, Janghwan Lee, Sukjin Hong, et al. RILQ: Rank-Insensitive LoRA-based Quantization Error Compensation for Boosting 2-bit Large Language Model Accuracy. AAAI, 2025. arXiv:2412.01129.

[7] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR, 2023. arXiv:2210.17323.

[8] Yanlong Zhao, Xiaoyuan Cheng, Huihang Liu, et al. When Lower Reconstruction Loss Hurts: Distributionally Robust Refinement for Low-Bit LLM Quantization. arXiv preprint, 2026. arXiv:2610.11226.

[9] Alireza Dadgarnia, Soroush Tabesh, Mahdi Nikdan, et al. GSQ: Highly-Accurate Low-Precision Scalar Quantization for LLMs via Gumbel-Softmax Sampling. arXiv preprint, 2026. arXiv:2604.18556.

[10] Yongge Ma, Guoan Wang, Feiyu Wang, et al. ReQuant: Fixed-Grid Discrete Refinement for Post-Training Quantization. arXiv preprint, 2026. arXiv:2608.07019.

[11] Gunjun Lee, Sehwan Son, Younjoo Lee, et al. SchurQuant: Groupwise Discrete Optimization for Layer-Wise LLM Quantization. arXiv preprint, 2026. arXiv:2608.15567.

[12] Seobin Song, Geonho Lee, Janghwan Lee, Jungwook Choi. Align, Then Correct: Training-Free Two-Stage Low-Rank Compensation for Extremely Quantized Large Language Models. arXiv preprint, 2026. arXiv:2610.08164.

[13] Miloš Nikolić, Ali Hadi Zadeh, Enrique Torres Sanchez, Andreas Moshovos. Displacement Is Not Direction: Evaluating Fidelity Metrics for Quantized LLM Deployment. arXiv preprint, 2026. arXiv:2606.19558.

[14] Yanxia Deng, Aozhong Zhang, Selcuk Gurses, et al. CLoQ: Enhancing Fine-Tuning of Quantized LLMs via Calibrated LoRA Initialization. Transactions on Machine Learning Research (TMLR), 2025. arXiv:2501.18475.

[15] Xin Wang, Yu Zheng, Zhongwei Wan, Mi Zhang. SVD-LLM: Truncation-aware Singular Value Decomposition for Large Language Model Compression. ICLR, 2025. arXiv:2403.07378.

[16] Nicolas Gillis, François Glineur. Low-Rank Matrix Approximation with Weights or Missing Data is NP-hard. SIAM J. Matrix Anal. & Appl. 32 (4), pp. 1149-1165, 2011. arXiv:1012.0197.

[17] Yuhang Li, Donghyun Lee, Ruokai Yin, Priyadarshini Panda. Optimal Brain Decomposition for Accurate LLM Low-Rank Approximation. arXiv preprint, 2026. arXiv:2604.00821.

[18] Nathan Halko, Per-Gunnar Martinsson, Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. SIAM Rev., Survey and Review section, Vol. 53, num. 2, pp. 217-288, June 2011. arXiv:0909.4061.

[19] Bowen Ping, Shuo Wang, Hanqing Wang, et al. Delta-CoMe: Training-Free Delta-Compression with Mixed-Precision for Large Language Models. NeurIPS, 2024. arXiv:2406.08903.

[20] Shihao Zhang, Rayan Saab. GPTQ-intrinsic LoRA: A Near-optimal Algorithm for Low-precision Quantization with Low-rank Adaptation. arXiv preprint, 2026. arXiv:2606.01412.

[21] Yixiao Li, Yifan Yu, Chen Liang, et al. LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models. ICLR, 2024. arXiv:2310.08659.

[22] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, Ramachandran Ramjee. Accuracy is Not All You Need. NeurIPS, 2024. arXiv:2407.09141.

[23] Yamato Arai, Yuma Ichikawa. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. NeurIPS, 2025. arXiv:2504.09629.

[24] Everlyn Asiko Chimoto, Mostafa Elhoushi, Bruce A. Bassett. Calibrating Beyond English: Language Diversity for Better Quantized Multilingual LLM. EACL, 2026. arXiv:2601.18306.

[25] Karl Audun Borgersen, Morten Goodwin. English K_Quantization of LLMs Does Not Disproportionately Diminish Multilingual Performance. arXiv preprint, 2025. arXiv:2503.03592.

[26] Vladimir Malinovskii, Andrei Panferov, Ivan Ilin, et al. Pushing the Limits of Large Language Model Quantization via the Linearity Theorem. arXiv preprint, 2024. arXiv:2411.17525.

[27] Sungbin Shin, Wonpyo Park, Jaeho Lee, Namhoon Lee. Rethinking Pruning Large Language Models: Benefits and Pitfalls of Reconstruction Error Minimization. EMNLP, 2024. arXiv:2406.15524.

[28] Wenhua Cheng, Weiwei Zhang, Haihao Shen, et al. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. Findings of EMNLP, 2024. arXiv:2309.05516.

[29] Albert Tseng, Zhaofeng Sun, Christopher De Sa. Model-Preserving Adaptive Rounding. ICML, 2026. arXiv:2505.22988.

[30] Wenya Yu, Chao Zhang, Li Wang, et al. ProjQ: Project-and-Quantize for Adapter-Aware LLM Compression. ICML, 2026. arXiv:2606.00494.

[31] Xinyu Wang, Sicheng Lyu, Xiao-Wen Chang. JARQ: Joint Alternating Refinement for Quantization. arXiv preprint, 2026. arXiv:2609.38599.

[32] Tim Dettmers, Mike Lewis, Younes Belkada, Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS, 2022. arXiv:2208.07339.

[33] Mingjie Sun, Xinlei Chen, J. Zico Kolter, Zhuang Liu. Massive Activations in Large Language Models. COLM, 2024. arXiv:2402.17762.

[34] Mengxia Yu, De Wang, Qi Shan, et al. The Super Weight in Large Language Models. arXiv preprint, 2024. arXiv:2411.07191.

[35] I Kennedy, T Kennedy. Quantization Error Is Spectrally Flat: A Single Random Probe Is a Calibrated, Data-Free Sensitivity Estimator, with Application to Budget-Targeted Mixed-Precision Quantization. arXiv preprint, 2026. arXiv:2609.33923.

[36] Yifan Liu. Task-Phase-Conditioned Expert Routing Is Not a Reliable Proxy for Quantization Sensitivity in MoE FFN Tensors. arXiv preprint, 2026. arXiv:2610.20826.

[37] Yoonjun Cho, Dongjae Jeon, Soeun Kim, et al. Preserve-Then-Quantize: Balancing Rank Budgets for Quantization Error Reconstruction in LLMs. ICML, 2026. arXiv:2602.02001.

[38] Selim An, Il hong Suh, Yeseong Kim. GlowQ: Group-Shared LOw-Rank Approximation for Quantized LLMs. ICLR, 2026. arXiv:2603.25385.

[39] Michael Helcig, Dan Alistarh. Model Compression with Exact Budget Constraints via Riemannian Manifolds. arXiv preprint, 2026. arXiv:2605.00649.

[40] Zehao Liu, Chuangchuang Fang, Yang Ren. Activation-Weighted Seeded Residual Coding for Low-Bit LLM Weight Repair. arXiv preprint, 2026. arXiv:2608.23144.

[41] Shuaiting Li, Juncan Deng, Kedong Xu, et al. Rethinking Residual Errors in Compensation-based LLM Quantization. ICLR, 2026. arXiv:2604.07955.

[42] Xiaodong Chen, Mingming Ha, Zhenzhong Lan, et al. MoBE: Mixture-of-Basis-Experts for Compressing MoE-based LLMs. arXiv preprint, 2025. arXiv:2508.05257.

[43] Qian Zhang, Yaoming Li, Zhewen Tan, et al. REAL-Q: E2E LLM Quantization via Dynamic Gradient Descent. arXiv preprint, 2026. arXiv:2609.00049.

[44] Meng Xin, Sweta Priyadarshi, Jingyu Xin, et al. Quantization-Aware Distillation for NVFP4 Inference Accuracy Recovery. arXiv preprint, 2026. arXiv:2601.20088.

[45] Michael Helcig, Eldar Kurtic, Dan Alistarh. Statistically-Lossless Quantization of Large Language Models. arXiv preprint, 2026. arXiv:2605.02404.

[46] Weili Xu, Jisen Li, Yuqing Jian, et al. QATFactory: A Versatile, Deployment-Aligned Framework for Quantization-aware Training and Distillation of LLMs. arXiv preprint, 2026. arXiv:2609.39223.

[47] Yuhe Hu. Quantization Effects on Tool-Failure Recovery Vary Across Prompts and Evaluation Designs. NeurIPS Workshop, 2026. arXiv:2610.07781.

[48] Aditi Patodiya. Same Request, Different Answer: Quantization Amplifies Cache-Induced Divergence in LLM Serving. arXiv preprint, 2026. arXiv:2609.04748.

[49] Lequn Chen, Zihao Ye, Yongji Wu, et al. Punica: Multi-Tenant LoRA Serving. MLSys, 2024. arXiv:2310.18547.

[50] Qiyang Li, Rui Kong, Yuchen Li, et al. AdaFuse: Accelerating Dynamic Adapter Inference via Token-Level Pre-Gating and Fused Kernel Optimization. AAAI, 2026. arXiv:2603.11873.

[51] Wenhao Wu, Zishan Shao, Kangning Cui, et al. FlashSVD v1.5: Making Low-Rank Transformers Inference Actually Fast. arXiv preprint, 2026. arXiv:2605.08314.

[52] Muyang Li, Yujun Lin, Zhekai Zhang, et al. SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models. ICLR, 2025. arXiv:2411.05007.

[53] Xinhao Cheng, Zhihao Zhang, Yu Zhou, et al. MPK: A Compiler and Runtime for Mega-Kernelizing Tensor Programs. OSDI, 2026. arXiv:2512.22219.

[54] Albert Tseng, Jerry Chee, Qingyao Sun, et al. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. ICML, 2024. arXiv:2402.04396.

[55] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, et al. Extreme Compression of Large Language Models via Additive Quantization. ICML, 2024. arXiv:2401.06118.

[56] Albert Tseng, Qingyao Sun, David Hou, Christopher De Sa. QTIP: Quantization with Trellises and Incoherence Processing. NeurIPS, 2024. arXiv:2406.11235.

[57] Oliver Sieberling, Denis Kuznedelev, Eldar Kurtic, Dan Alistarh. EvoPress: Accurate Dynamic Model Compression via Evolutionary Search. arXiv preprint, 2024. arXiv:2410.14649.

[58] Amogh Singh. Certifying Compressed Language Models: An Audit and a Statistical Toolkit. arXiv preprint, 2026. arXiv:2608.15046.

[59] Beatriz Almeida Felicio. How Divergence Becomes Decision Flips in Compressed Language Models. arXiv preprint, 2026. arXiv:2610.00694.

[60] Vage Egiazarian, Roberto L. Castro, Denis Kuznedelev, et al. Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization. ICLR, 2026. arXiv:2509.23202.

[61] Yanlong Zhao, Xiaoyuan Cheng, Huihang Liu, et al. Saliency-Aware Regularized Quantization Calibration for Large Language Models. arXiv preprint, 2026. arXiv:2605.05693.

[62] Zakhar Shumaylov, Vasileios Tsiaras, Yannis Stylianou. On Information Geometry and Iterative Optimization in Model Compression: Operator Factorization. arXiv preprint, 2025. arXiv:2507.09428.

[63] Meyer Scetbon, James Hensman. Low-Rank Correction for Quantized LLMs. arXiv preprint, 2024. arXiv:2412.07902.

[64] Utkarsh Saxena, Sayeh Sharify, Kaushik Roy, Xin Wang. ResQ: Mixed-Precision Quantization of Large Language Models with Low-Rank Residuals. ICML, 2025. arXiv:2412.14363.

[65] Hongyaoxing Gu, Lijuan Hu, Shuzi Niu, Fangfang Liu. FLRQ: Faster LLM Quantization with Flexible Low-Rank Matrix Sketching. AAAI, 2026. arXiv:2601.05684.

[66] Haodong Wang, Junjie Liu, Zicong Hong, et al. TwinQuant: Learnable Subspace Decomposition for 4-Bit LLM Quantization. ICML, 2026. arXiv:2606.01556.

[67] Jinuk Kim, Marwa El Halabi, Wonpyo Park, et al. GuidedQuant: Large Language Model Quantization via Exploiting End Loss Guidance. ICML, 2025. arXiv:2505.07004.

[68] Donghyun Lee, Yuhang Li, Ruokai Yin, Priyadarshini Panda. KronQ: LLM Quantization via Kronecker-Factored Hessian. COLM, 2026. arXiv:2607.07964.

[69] YiFeng Wang, Zhun Sun, Keisuke Sakaguchi. Technical Report: Activation Residual Hessian Quantization (ARHQ) for Low-Bit LLM Quantization. arXiv preprint, 2026. arXiv:2605.00140.

[70] Jundong Hu, Shekar Ramachandran. The Structure of Quantization Damage in LLMs: Why the Next Bit Should Be Spent Globally. arXiv preprint, 2026. arXiv:2609.01587.

[71] Han Guo, Philip Greengard, Eric P. Xing, Yoon Kim. LQ-LoRA: Low-rank Plus Quantized Matrix Decomposition for Efficient Language Model Finetuning. ICLR, 2024. arXiv:2311.12023.

[72] Beichen Huang, Yueming Yuan, Zelei Shao, Minjia Zhang. MiLo: Efficient Quantized MoE Inference with Mixture of Low-Rank Compensators. MLSys, 2025. arXiv:2504.02658.

[73] Hongyaoxing Gu, Xinzhe Chen, Lijuan Hu, Fangfang Liu. TileQ: Efficient Low-Rank Quantization of Mixture-of-Experts with 2D Tiling. ICML, 2026. arXiv:2605.09281.

[74] Zhenyu Liu, Yunzhen Liu, Zehao Fan, et al. Bandwidth-Efficient Adaptive Mixture-of-Experts via Low-Rank Compensation. arXiv preprint, 2025. arXiv:2512.17073.

[75] Hongyuan Liu, Yawei Li, Zhiqiang Que, et al. SPEAR: A System for Post-Quantization Error-Adaptive Recovery Enabling Efficient Low-Bit LLM Serving. arXiv preprint, 2026. arXiv:2606.11244.

[76] Yeonhong Park, Jake Hyun, Hojoon Kim, Jae W. Lee. DecDEC: A Systems Approach to Advancing Low-Bit LLM Quantization. OSDI, 2025. arXiv:2412.20185.

[77] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, Danqi Chen. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. ICLR, 2024. arXiv:2310.06694.

[78] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, et al. Compact Language Models via Pruning and Knowledge Distillation. NeurIPS, 2024. arXiv:2407.14679.